

UAS Deep Learning

Nomor 1 (LSTM Time Series Prediction)

Name: Syaquila Najwa Amalia Hambali

NIM: 2802488481

Video Explanation

Link video explanation: <https://youtu.be/7netmugwSJ8?si=mfETwFyQmIIskDjC>

1a. Data Exploration and Preprocessing

Pada bagian ini dilakukan eksplorasi dan preprocessing data saham sebelum digunakan pada model LSTM. Dataset yang digunakan terdiri dari dua data time series saham, yaitu FB dan IBM. Karena data saham merupakan data time series, maka urutan tanggal harus dipertahankan dan data tidak boleh diacak.

Sesuai instruksi soal, kolom yang digunakan adalah Date dan Close. Kolom Date digunakan sebagai penanda waktu perdagangan, sedangkan kolom Close digunakan sebagai harga penutupan yang akan diprediksi oleh model.

Data test diambil dari periode satu tahun terakhir. Data sebelum periode test digunakan sebagai training data, kemudian training data tersebut dibagi kembali menjadi 90% train set dan 10% validation set. Pembagian dilakukan secara manual berdasarkan urutan waktu agar tidak terjadi data leakage.

Setelah itu, data diubah menjadi bentuk supervised learning menggunakan window size sebesar 5 dan horizon sebesar 1. Artinya, harga penutupan selama 5 hari sebelumnya digunakan sebagai input untuk memprediksi harga penutupan pada 1 hari berikutnya.

```
In [1]: # import library  
  
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
```

In [2]: *# Load Dataset*

```
fb = pd.read_csv("FB.csv")
ibm = pd.read_csv("IBM.csv")

print("FB dataset shape:", fb.shape)
print("IBM dataset shape:", ibm.shape)

display(fb.head())
display(ibm.head())
```

FB dataset shape: (1980, 7)
IBM dataset shape: (14663, 7)

	Date	Open	High	Low	Close	Adj Close	Volume
0	2012-05-18	42.049999	45.000000	38.000000	38.230000	38.230000	573576400
1	2012-05-21	36.529999	36.660000	33.000000	34.029999	34.029999	168192700
2	2012-05-22	32.610001	33.590000	30.940001	31.000000	31.000000	101786600
3	2012-05-23	31.370001	32.500000	31.360001	32.000000	32.000000	73600000
4	2012-05-24	32.950001	33.209999	31.770000	33.029999	33.029999	50237200

	Date	Open	High	Low	Close	Adj Close	Volume
0	1962-01-02	7.713333	7.713333	7.626667	7.626667	0.618153	387200
1	1962-01-03	7.626667	7.693333	7.626667	7.693333	0.623556	288000
2	1962-01-04	7.693333	7.693333	7.613333	7.616667	0.617343	256000
3	1962-01-05	7.606667	7.606667	7.453333	7.466667	0.605185	363200
4	1962-01-08	7.460000	7.460000	7.266667	7.326667	0.593837	544000

Berdasarkan hasil load dataset, data saham FB memiliki 1.980 baris dan 7 kolom, sedangkan data saham IBM memiliki 14.663 baris dan 7 kolom. Kedua dataset memiliki struktur kolom yang sama, yaitu Date, Open, High, Low, Close, Adj Close, dan Volume.

Namun, sesuai instruksi soal, kolom yang digunakan untuk membangun model LSTM hanya Date dan Close. Kolom Date digunakan untuk menjaga urutan waktu, sedangkan kolom Close digunakan sebagai nilai harga penutupan yang akan diprediksi.

```
In [3]: # Cek Struktur Data

print("FB Dataset Information")
fb.info()

print("\nIBM Dataset Information")
ibm.info()
```

FB Dataset Information

<class 'pandas.DataFrame'>

RangeIndex: 1980 entries, 0 to 1979

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	Date	1980 non-null	str
1	Open	1980 non-null	float64
2	High	1980 non-null	float64
3	Low	1980 non-null	float64
4	Close	1980 non-null	float64
5	Adj Close	1980 non-null	float64
6	Volume	1980 non-null	int64

dtypes: float64(5), int64(1), str(1)

memory usage: 127.7 KB

IBM Dataset Information

<class 'pandas.DataFrame'>

RangeIndex: 14663 entries, 0 to 14662

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	Date	14663 non-null	str
1	Open	14663 non-null	float64
2	High	14663 non-null	float64
3	Low	14663 non-null	float64
4	Close	14663 non-null	float64
5	Adj Close	14663 non-null	float64
6	Volume	14663 non-null	int64

dtypes: float64(5), int64(1), str(1)

memory usage: 945.2 KB

In [4]: *# Check Missing Values*

```
print("Missing values in FB dataset:")
display(fb.isnull().sum())

print("Missing values in IBM dataset:")
display(ibm.isnull().sum())
```

Missing values in FB dataset:

```
Date          0
Open          0
High          0
Low           0
Close         0
Adj Close     0
Volume        0
dtype: int64
Missing values in IBM dataset:
Date          0
Open          0
High          0
Low           0
Close         0
Adj Close     0
Volume        0
dtype: int64
```

Berdasarkan hasil pengecekan struktur data, dataset FB dan IBM memiliki kolom yang sama. Kolom Date masih bertipe object/string sehingga perlu diubah menjadi format datetime. Kolom Close sudah bertipe numerik sehingga dapat digunakan untuk proses normalisasi dan pembuatan data sequence LSTM.

Berdasarkan hasil pengecekan missing value, tidak terdapat nilai kosong pada dataset FB maupun IBM. Oleh karena itu, tidak diperlukan proses imputasi pada tahap preprocessing.

```
In [5]: # Select Date and Close Columns

fb_stock = fb[["Date", "Close"]].copy()
ibm_stock = ibm[["Date", "Close"]].copy()

fb_stock["Date"] = pd.to_datetime(fb_stock["Date"])
ibm_stock["Date"] = pd.to_datetime(ibm_stock["Date"])

fb_stock = fb_stock.sort_values("Date").reset_index(drop=True)
ibm_stock = ibm_stock.sort_values("Date").reset_index(drop=True)

display(fb_stock.head())
display(ibm_stock.head())
```

	Date	Close
0	2012-05-18	38.230000
1	2012-05-21	34.029999
2	2012-05-22	31.000000
3	2012-05-23	32.000000
4	2012-05-24	33.029999

	Date	Close
0	1962-01-02	7.626667
1	1962-01-03	7.693333
2	1962-01-04	7.616667
3	1962-01-05	7.466667
4	1962-01-08	7.326667

Pada tahap ini, hanya kolom Date dan Close yang dipilih dari dataset FB dan IBM. Kolom Date diubah menjadi format datetime agar dapat diproses sebagai data time series, kemudian data diurutkan berdasarkan tanggal. Kolom Close digunakan sebagai variabel utama karena harga penutupan menjadi nilai yang akan diprediksi oleh model LSTM.

```
In [6]: # Check Date Range

print("FB date range:")
print(fb_stock["Date"].min(), "to", fb_stock["Date"].max())

print("\nIBM date range:")
print(ibm_stock["Date"].min(), "to", ibm_stock["Date"].max())
```

```
FB date range:
2012-05-18 00:00:00 to 2020-04-01 00:00:00
```

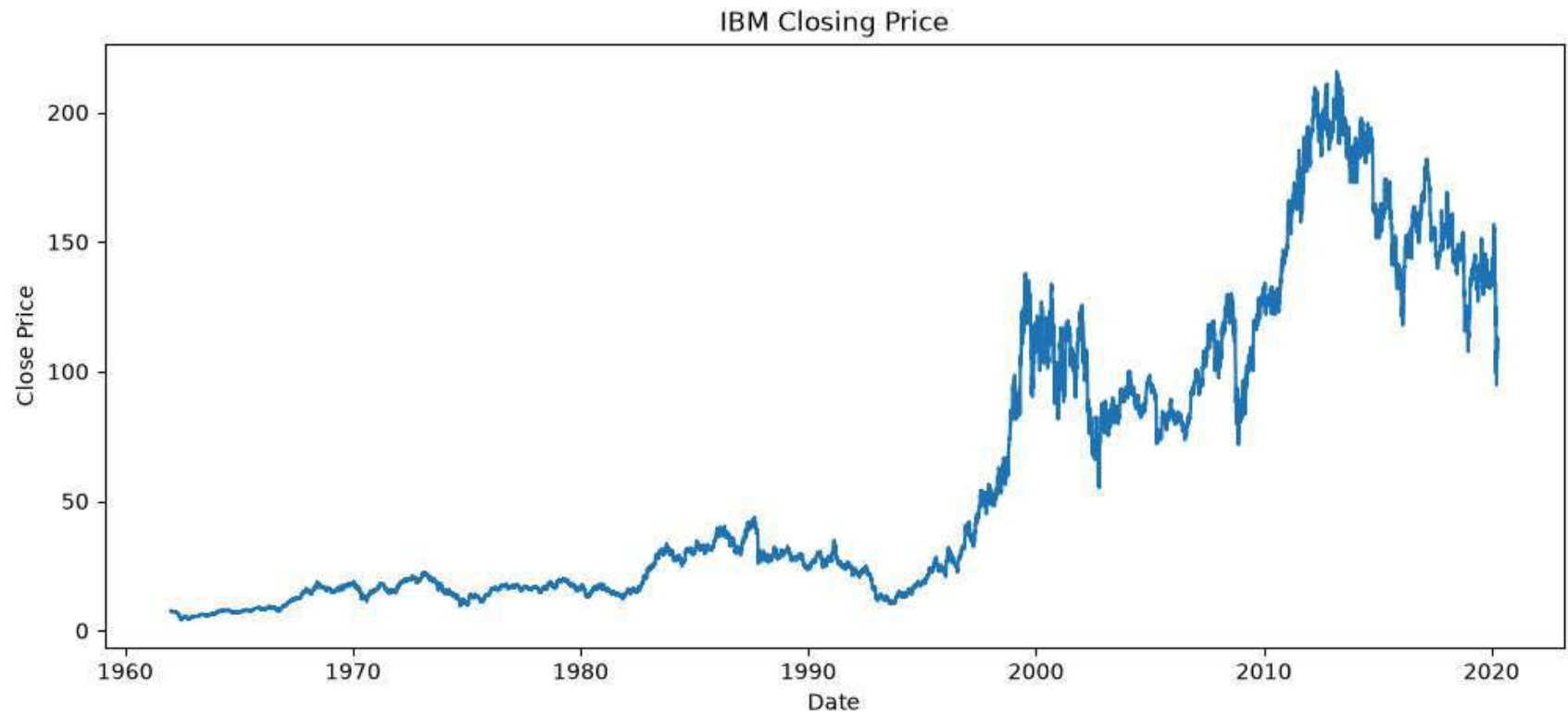
```
IBM date range:
1962-01-02 00:00:00 to 2020-04-01 00:00:00
```

In [7]: *# Visualize Closing Price*

```
plt.figure(figsize=(12, 5))
plt.plot(fb_stock["Date"], fb_stock["Close"])
plt.title("FB Closing Price")
plt.xlabel("Date")
plt.ylabel("Close Price")
plt.show()

plt.figure(figsize=(12, 5))
plt.plot(ibm_stock["Date"], ibm_stock["Close"])
plt.title("IBM Closing Price")
plt.xlabel("Date")
plt.ylabel("Close Price")
plt.show()
```





Berdasarkan hasil pengecekan periode data, saham FB memiliki data dari 18 Mei 2012 sampai 1 April 2020, sedangkan saham IBM memiliki data dari 2 Januari 1962 sampai 1 April 2020. Perbedaan periode ini menunjukkan bahwa jumlah data IBM jauh lebih banyak dibandingkan FB.

Visualisasi harga penutupan menunjukkan bahwa kedua saham memiliki pola pergerakan harga yang berubah dari waktu ke waktu. Saham FB cenderung mengalami kenaikan harga sejak awal periode hingga sekitar tahun 2018–2020, meskipun terdapat beberapa penurunan tajam. Sementara itu, saham IBM memiliki histori yang lebih panjang dan menunjukkan beberapa fase kenaikan serta penurunan harga yang cukup besar.

Karena data memiliki urutan waktu, proses pembagian data tidak dilakukan secara acak. Data satu tahun terakhir digunakan sebagai test set karena merepresentasikan periode masa depan yang ingin diprediksi. Sementara itu, data sebelum periode test digunakan sebagai training data, kemudian dibagi kembali menjadi train set dan validation set.

```
In [8]: # Create LSTM Dataset with Window Size 5 and One-Year Test Set
```

```

def create_lstm_dataset(stock_data, window_size=5):
    stock_data = stock_data.copy()
    stock_data["Date"] = pd.to_datetime(stock_data["Date"])
    stock_data = stock_data.sort_values("Date").reset_index(drop=True)

    # Test set uses the last one-year period
    test_start_date = stock_data["Date"].max() - pd.DateOffset(years=1)

    # Target date starts after window size
    target_dates = stock_data["Date"].iloc[window_size:].reset_index(drop=True)

    # Count train-validation and test samples based on target date
    train_val_count = (target_dates < test_start_date).sum()
    train_count = int(train_val_count * 0.9)

    # Fit MinMaxScaler only on train set to avoid data leakage
    train_last_row_index = window_size + train_count - 1

    scaler = MinMaxScaler(feature_range=(0, 1))
    scaler.fit(stock_data.loc[:train_last_row_index, ["Close"]])

    close_scaled = scaler.transform(stock_data[["Close"]])

    X = []
    y = []

    for i in range(window_size, len(close_scaled)):
        X.append(close_scaled[i-window_size:i, 0])
        y.append(close_scaled[i, 0])

    X = np.array(X)
    y = np.array(y)

    X = X.reshape(X.shape[0], X.shape[1], 1)

    X_train = X[:train_count]
    y_train = y[:train_count]

    X_val = X[train_count:train_val_count]
    y_val = y[train_count:train_val_count]

    X_test = X[train_val_count:]

```

```

y_test = y[train_val_count:]

split_info = pd.DataFrame({
    "Split": ["Train", "Validation", "Test"],
    "Start Date": [
        target_dates.iloc[0],
        target_dates.iloc[train_count],
        target_dates.iloc[train_val_count]
    ],
    "End Date": [
        target_dates.iloc[train_count - 1],
        target_dates.iloc[train_val_count - 1],
        target_dates.iloc[-1]
    ],
    "Samples": [
        len(X_train),
        len(X_val),
        len(X_test)
    ]
})

return scaler, X_train, X_val, X_test, y_train, y_val, y_test, split_info

```

In [9]: *# Apply Preprocessing to FB and IBM*

```

window_size = 5

fb_scaler, X_train_fb, X_val_fb, X_test_fb, y_train_fb, y_val_fb, y_test_fb, fb_split_info = create_lstm_dataset(
    fb_stock, window_size
)

ibm_scaler, X_train_ibm, X_val_ibm, X_test_ibm, y_train_ibm, y_val_ibm, y_test_ibm, ibm_split_info = create_lstm_dataset(
    ibm_stock, window_size
)

print("FB X_train shape:", X_train_fb.shape)
print("FB X_val shape :", X_val_fb.shape)
print("FB X_test shape :", X_test_fb.shape)
print("FB y_train shape:", y_train_fb.shape)
print("FB y_val shape :", y_val_fb.shape)
print("FB y_test shape :", y_test_fb.shape)

```

```

print("\nIBM X_train shape:", X_train_ibm.shape)
print("IBM X_val shape  :", X_val_ibm.shape)
print("IBM X_test shape :", X_test_ibm.shape)
print("IBM y_train shape:", y_train_ibm.shape)
print("IBM y_val shape  :", y_val_ibm.shape)
print("IBM y_test shape :", y_test_ibm.shape)

```

```

FB X_train shape: (1548, 5, 1)
FB X_val shape  : (173, 5, 1)
FB X_test shape : (254, 5, 1)
FB y_train shape: (1548,)
FB y_val shape  : (173,)
FB y_test shape : (254,)

```

```

IBM X_train shape: (12963, 5, 1)
IBM X_val shape  : (1441, 5, 1)
IBM X_test shape : (254, 5, 1)
IBM y_train shape: (12963,)
IBM y_val shape  : (1441,)
IBM y_test shape : (254,)

```

```

In [10]: print("\nFB Split Information")
display(fb_split_info)

print("\nIBM Split Information")
display(ibm_split_info)

```

FB Split Information

	Split	Start Date	End Date	Samples
0	Train	2012-05-25	2018-07-20	1548
1	Validation	2018-07-23	2019-03-29	173
2	Test	2019-04-01	2020-04-01	254

IBM Split Information

	Split	Start Date	End Date	Samples
0	Train	1962-01-09	2013-07-09	12963
1	Validation	2013-07-10	2019-03-29	1441
2	Test	2019-04-01	2020-04-01	254

Setelah dilakukan normalisasi dan pembuatan sequence, data berhasil diubah menjadi format yang sesuai untuk input LSTM, yaitu tiga dimensi dengan bentuk (samples, timesteps, features). Nilai timesteps adalah 5 karena window size yang digunakan sebesar 5, sedangkan features bernilai 1 karena hanya kolom Close yang digunakan.

Berdasarkan hasil split data, test set diambil dari periode satu tahun terakhir, yaitu 1 April 2019 sampai 1 April 2020. Hal ini sesuai dengan instruksi soal yang menyatakan bahwa data test harus mencakup periode satu tahun terakhir.

Pada saham FB, data dibagi menjadi 1.548 sequence untuk train set, 173 sequence untuk validation set, dan 254 sequence untuk test set. Pada saham IBM, data dibagi menjadi 12.963 sequence untuk train set, 1.441 sequence untuk validation set, dan 254 sequence untuk test set. Jumlah data train dan validation IBM lebih besar karena histori data IBM jauh lebih panjang dibandingkan FB.

Pembagian data dilakukan secara manual berdasarkan urutan waktu dan tidak diacak karena dataset merupakan data time series. Data sebelum periode test digunakan sebagai training data, kemudian training data tersebut dibagi kembali menjadi 90% train set dan 10% validation set. Train set digunakan untuk melatih model, validation set digunakan untuk memantau performa selama training, sedangkan test set digunakan untuk evaluasi akhir menggunakan RMSE, MAE, dan MAPE.

Tanggal awal train set tidak dimulai dari tanggal pertama pada dataset karena data sudah diubah menjadi sequence dengan window size sebesar 5. Artinya, lima data pertama digunakan sebagai input awal untuk memprediksi target pertama atau horizon 1.

```
In [11]: # Check MinMax Scaling Result

def check_minmax_scaling(stock_data, scaler, stock_name):
    scaled_close = scaler.transform(stock_data[["Close"]])

    summary = pd.DataFrame({
        "Stock": [stock_name],
        "Original Close Min": [stock_data["Close"].min()],
        "Original Close Max": [stock_data["Close"].max()],
```

```

        "Scaled Close Min": [scaled_close.min()],
        "Scaled Close Max": [scaled_close.max()]
    })

    return summary

fb_minmax_summary = check_minmax_scaling(fb_stock, fb_scaler, "FB")
ibm_minmax_summary = check_minmax_scaling(ibm_stock, ibm_scaler, "IBM")

minmax_summary = pd.concat([fb_minmax_summary, ibm_minmax_summary], ignore_index=True)
minmax_summary

```

```

Out[11]:

```

	Stock	Original Close Min	Original Close Max	Scaled Close Min	Scaled Close Max
0	FB	17.73	223.229996	0.0	1.068865
1	IBM	4.08	215.800003	0.0	1.000000

MinMax Scaling Result

Pada tahap preprocessing, nilai Close dinormalisasi menggunakan MinMaxScaler. Scaler hanya di-fit pada train set untuk menghindari data leakage, sehingga informasi dari validation set dan test set tidak digunakan dalam proses fitting scaler.

Karena scaler hanya mempelajari nilai minimum dan maksimum dari train set, nilai pada validation set atau test set dapat berada di luar rentang 0 sampai 1 apabila harga pada periode tersebut lebih tinggi atau lebih rendah dibandingkan rentang harga pada train set. Hal ini masih wajar dan lebih aman dibandingkan melakukan fitting scaler pada seluruh dataset, karena fitting pada seluruh dataset dapat menyebabkan data leakage.

Pada saham FB, nilai scaled close maksimum melebihi 1 karena scaler hanya di-fit pada train set. Jika terdapat harga pada validation atau test set yang lebih tinggi dari maksimum train set, hasil transformasi dapat menjadi lebih besar dari 1. Hal ini tidak menjadi masalah karena scaler memang hanya menggunakan informasi dari train set untuk menghindari data leakage.

1b. Baseline LSTM Model

Pada bagian ini dibuat model baseline LSTM untuk memprediksi harga penutupan saham FB dan IBM. Model baseline digunakan sebagai model awal yang nantinya akan dibandingkan dengan modified LSTM pada bagian berikutnya.

Sesuai instruksi soal, arsitektur baseline menggunakan satu layer LSTM dengan 50 units dan activation function ReLU. Layer akhir menggunakan satu node Perceptron atau Dense layer dengan 1 unit karena model hanya memprediksi satu nilai, yaitu harga Close pada hari berikutnya.

Model dilatih menggunakan train set dan dipantau menggunakan validation set. Test set tidak digunakan pada tahap training karena test set hanya digunakan untuk evaluasi akhir pada bagian 1d.

```
In [12]: # Import Deep Learning Libraries

import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, LSTM, Dense
from tensorflow.keras.optimizers import Adam

np.random.seed(42)
tf.random.set_seed(42)
```

```
In [13]: # Build Baseline LSTM Model

def build_baseline_lstm(input_shape):
    model = Sequential()
    model.add(Input(shape=input_shape))
    model.add(LSTM(units=50, activation="relu"))
    model.add(Dense(1))

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss="mse"
    )

    return model
```

Arsitektur baseline LSTM terdiri dari beberapa komponen utama:

- LSTM(units=50, activation="relu") digunakan untuk mempelajari pola historis dari data time series.
- Dense(1) digunakan sebagai output layer karena model hanya memprediksi satu nilai, yaitu harga Close pada hari berikutnya.
- Loss function yang digunakan adalah MSE karena permasalahan ini merupakan regresi.

- Optimizer yang digunakan adalah Adam dengan learning rate 0.001.

In [14]: `# Baseline LSTM for FB`

```
baseline_fb = build_baseline_lstm((X_train_fb.shape[1], X_train_fb.shape[2]))
baseline_fb.summary()
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuD NN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50)	10,400
dense (Dense)	(None, 1)	51

Total params: 10,451 (40.82 KB)

Trainable params: 10,451 (40.82 KB)

Non-trainable params: 0 (0.00 B)

Model baseline FB dilatih menggunakan X_train_fb dan y_train_fb. Validation set digunakan untuk memantau performa model selama training. Test set tidak digunakan pada tahap ini agar evaluasi akhir tetap objektif.

Model dilatih selama 50 epoch dengan batch size 32. Nilai loss yang digunakan adalah MSE, sehingga semakin kecil nilai loss menunjukkan bahwa prediksi model semakin mendekati nilai aktual pada skala data yang sudah dinormalisasi.

In [15]: `# Train Baseline LSTM for FB`

```
history_baseline_fb = baseline_fb.fit(
    X_train_fb,
    y_train_fb,
    validation_data=(X_val_fb, y_val_fb),
    epochs=50,
    batch_size=32,
    verbose=1
)
```

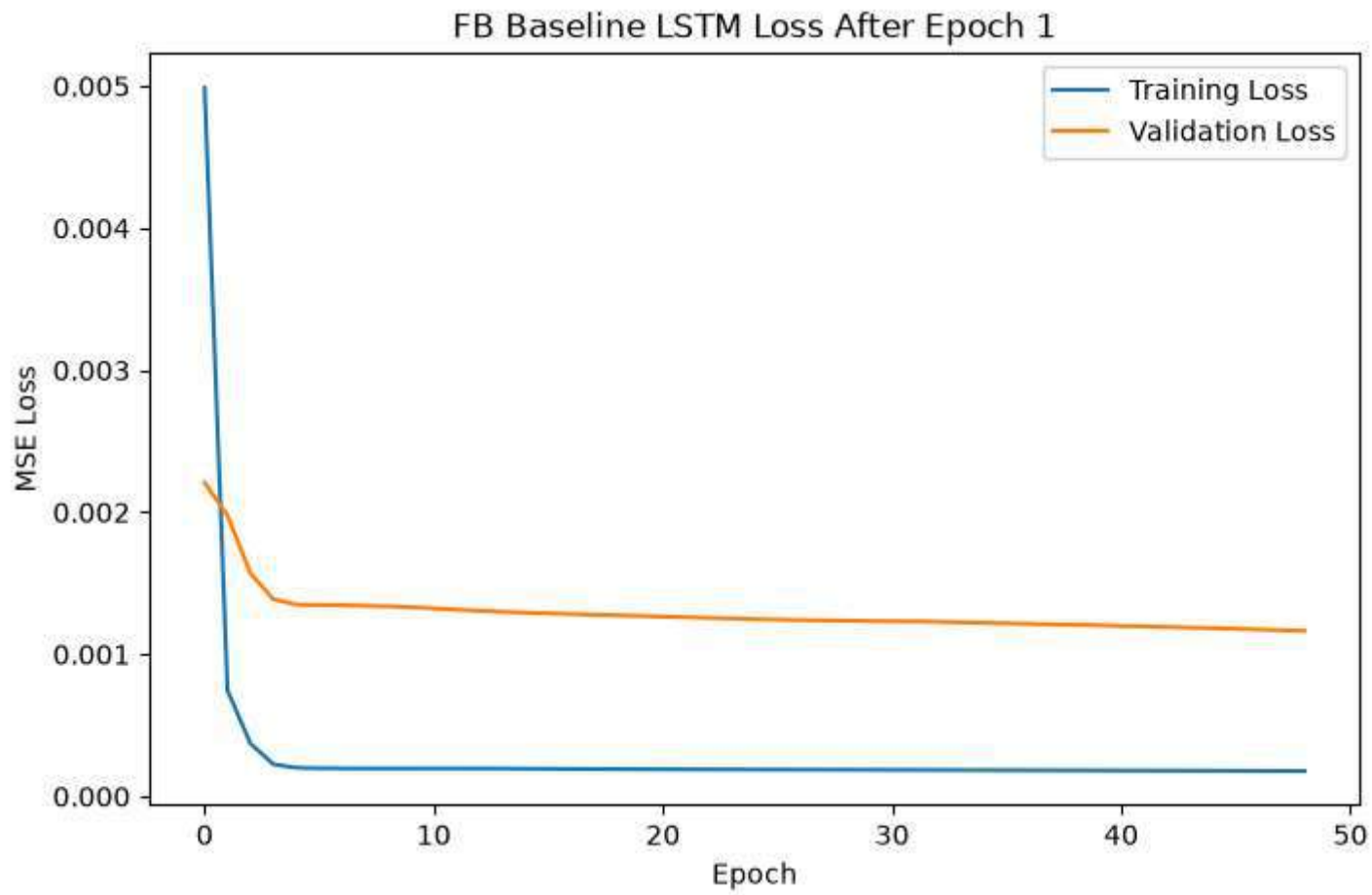
Epoch 1/50
49/49  2s 10ms/step - loss: 0.1123 - val_loss: 0.0012
Epoch 2/50
49/49  0s 4ms/step - loss: 0.0050 - val_loss: 0.0022
Epoch 3/50
49/49  0s 4ms/step - loss: 7.4408e-04 - val_loss: 0.0020
Epoch 4/50
49/49  0s 4ms/step - loss: 3.7118e-04 - val_loss: 0.0016
Epoch 5/50
49/49  0s 4ms/step - loss: 2.2551e-04 - val_loss: 0.0014
Epoch 6/50
49/49  0s 4ms/step - loss: 2.0055e-04 - val_loss: 0.0013
Epoch 7/50
49/49  0s 4ms/step - loss: 1.9760e-04 - val_loss: 0.0013
Epoch 8/50
49/49  0s 4ms/step - loss: 1.9603e-04 - val_loss: 0.0013
Epoch 9/50
49/49  0s 4ms/step - loss: 1.9565e-04 - val_loss: 0.0013
Epoch 10/50
49/49  0s 3ms/step - loss: 1.9557e-04 - val_loss: 0.0013
Epoch 11/50
49/49  0s 4ms/step - loss: 1.9566e-04 - val_loss: 0.0013
Epoch 12/50
49/49  0s 4ms/step - loss: 1.9577e-04 - val_loss: 0.0013
Epoch 13/50
49/49  0s 4ms/step - loss: 1.9579e-04 - val_loss: 0.0013
Epoch 14/50
49/49  0s 4ms/step - loss: 1.9546e-04 - val_loss: 0.0013
Epoch 15/50
49/49  0s 4ms/step - loss: 1.9493e-04 - val_loss: 0.0013
Epoch 16/50
49/49  0s 4ms/step - loss: 1.9417e-04 - val_loss: 0.0013
Epoch 17/50
49/49  0s 4ms/step - loss: 1.9335e-04 - val_loss: 0.0013
Epoch 18/50
49/49  0s 4ms/step - loss: 1.9247e-04 - val_loss: 0.0013
Epoch 19/50
49/49  0s 4ms/step - loss: 1.9175e-04 - val_loss: 0.0013
Epoch 20/50
49/49  0s 4ms/step - loss: 1.9100e-04 - val_loss: 0.0013
Epoch 21/50
49/49  0s 4ms/step - loss: 1.9037e-04 - val_loss: 0.0013

Epoch 22/50
49/49  0s 4ms/step - loss: 1.8985e-04 - val_loss: 0.0013
Epoch 23/50
49/49  0s 4ms/step - loss: 1.8935e-04 - val_loss: 0.0013
Epoch 24/50
49/49  0s 4ms/step - loss: 1.8897e-04 - val_loss: 0.0013
Epoch 25/50
49/49  0s 4ms/step - loss: 1.8859e-04 - val_loss: 0.0013
Epoch 26/50
49/49  0s 3ms/step - loss: 1.8832e-04 - val_loss: 0.0012
Epoch 27/50
49/49  0s 4ms/step - loss: 1.8801e-04 - val_loss: 0.0012
Epoch 28/50
49/49  0s 4ms/step - loss: 1.8775e-04 - val_loss: 0.0012
Epoch 29/50
49/49  0s 4ms/step - loss: 1.8742e-04 - val_loss: 0.0012
Epoch 30/50
49/49  0s 4ms/step - loss: 1.8712e-04 - val_loss: 0.0012
Epoch 31/50
49/49  0s 4ms/step - loss: 1.8667e-04 - val_loss: 0.0012
Epoch 32/50
49/49  0s 5ms/step - loss: 1.8623e-04 - val_loss: 0.0012
Epoch 33/50
49/49  0s 5ms/step - loss: 1.8580e-04 - val_loss: 0.0012
Epoch 34/50
49/49  0s 6ms/step - loss: 1.8535e-04 - val_loss: 0.0012
Epoch 35/50
49/49  0s 6ms/step - loss: 1.8486e-04 - val_loss: 0.0012
Epoch 36/50
49/49  0s 5ms/step - loss: 1.8434e-04 - val_loss: 0.0012
Epoch 37/50
49/49  0s 5ms/step - loss: 1.8389e-04 - val_loss: 0.0012
Epoch 38/50
49/49  0s 6ms/step - loss: 1.8348e-04 - val_loss: 0.0012
Epoch 39/50
49/49  0s 7ms/step - loss: 1.8303e-04 - val_loss: 0.0012
Epoch 40/50
49/49  0s 6ms/step - loss: 1.8255e-04 - val_loss: 0.0012
Epoch 41/50
49/49  0s 5ms/step - loss: 1.8200e-04 - val_loss: 0.0012
Epoch 42/50
49/49  0s 5ms/step - loss: 1.8155e-04 - val_loss: 0.0012

```
Epoch 43/50
49/49 ————— 0s 5ms/step - loss: 1.8107e-04 - val_loss: 0.0012
Epoch 44/50
49/49 ————— 0s 4ms/step - loss: 1.8064e-04 - val_loss: 0.0012
Epoch 45/50
49/49 ————— 0s 4ms/step - loss: 1.8021e-04 - val_loss: 0.0012
Epoch 46/50
49/49 ————— 0s 5ms/step - loss: 1.7976e-04 - val_loss: 0.0012
Epoch 47/50
49/49 ————— 0s 5ms/step - loss: 1.7937e-04 - val_loss: 0.0012
Epoch 48/50
49/49 ————— 0s 5ms/step - loss: 1.7896e-04 - val_loss: 0.0012
Epoch 49/50
49/49 ————— 0s 4ms/step - loss: 1.7850e-04 - val_loss: 0.0012
Epoch 50/50
49/49 ————— 0s 4ms/step - loss: 1.7802e-04 - val_loss: 0.0012
```

In [16]: *# Plot Training and Validation Loss for FB Baseline Model*

```
plt.figure(figsize=(8, 5))
plt.plot(history_baseline_fb.history["loss"][1:], label="Training Loss")
plt.plot(history_baseline_fb.history["val_loss"][1:], label="Validation Loss")
plt.title("FB Baseline LSTM Loss After Epoch 1")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.show()
```



Berdasarkan grafik loss baseline LSTM pada saham FB, training loss mengalami penurunan tajam pada epoch awal, kemudian mulai stabil pada nilai yang rendah. Validation loss juga mengalami penurunan, meskipun nilainya masih lebih tinggi dibandingkan training loss.

Pada hasil akhir, baseline LSTM untuk saham FB memperoleh final training loss sebesar 0.000178 dan final validation loss sebesar 0.001165. Perbedaan antara training loss dan validation loss menunjukkan adanya gap generalisasi, tetapi model masih dapat digunakan sebagai baseline karena validation loss tetap berada pada nilai yang rendah.

Hasil ini digunakan sebagai pembanding awal sebelum dilakukan modifikasi arsitektur LSTM.

```
In [17]: # Baseline LSTM for IBM

baseline_ibm = build_baseline_lstm((X_train_ibm.shape[1], X_train_ibm.shape[2]))

baseline_ibm.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 50)	10,400
dense_1 (Dense)	(None, 1)	51

Total params: 10,451 (40.82 KB)

Trainable params: 10,451 (40.82 KB)

Non-trainable params: 0 (0.00 B)

Model baseline IBM dilatih menggunakan X_train_ibm dan y_train_ibm. Validation set digunakan untuk memantau performa model selama training, sedangkan test set tetap disimpan untuk evaluasi akhir.

Arsitektur baseline IBM sama seperti baseline FB, yaitu menggunakan satu layer LSTM dengan 50 units dan satu Dense output layer. Hal ini dilakukan agar baseline model pada kedua saham memiliki struktur yang konsisten dan dapat dibandingkan dengan modified model.

```
In [18]: # Train Baseline LSTM for IBM

history_baseline_ibm = baseline_ibm.fit(
    X_train_ibm,
    y_train_ibm,
    validation_data=(X_val_ibm, y_val_ibm),
    epochs=50,
    batch_size=32,
    verbose=1
)
```

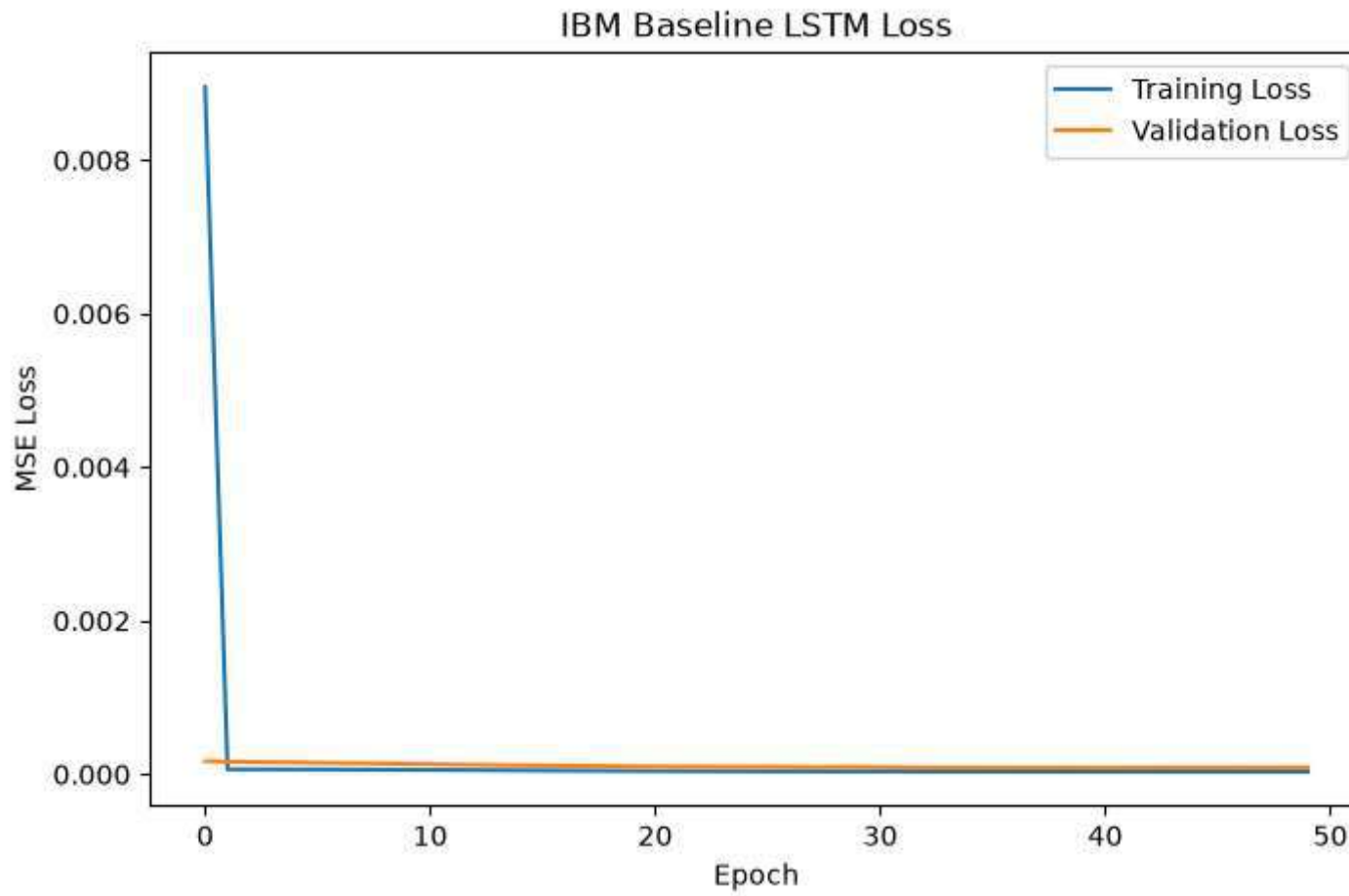
Epoch 1/50
406/406 ————— 3s 4ms/step - loss: 0.0090 - val_loss: 1.7033e-04
Epoch 2/50
406/406 ————— 1s 3ms/step - loss: 6.4232e-05 - val_loss: 1.6367e-04
Epoch 3/50
406/406 ————— 1s 3ms/step - loss: 6.3199e-05 - val_loss: 1.5938e-04
Epoch 4/50
406/406 ————— 1s 3ms/step - loss: 6.2596e-05 - val_loss: 1.5566e-04
Epoch 5/50
406/406 ————— 1s 3ms/step - loss: 6.2056e-05 - val_loss: 1.5234e-04
Epoch 6/50
406/406 ————— 1s 3ms/step - loss: 6.1381e-05 - val_loss: 1.4884e-04
Epoch 7/50
406/406 ————— 1s 3ms/step - loss: 6.0727e-05 - val_loss: 1.4585e-04
Epoch 8/50
406/406 ————— 1s 3ms/step - loss: 5.9882e-05 - val_loss: 1.4240e-04
Epoch 9/50
406/406 ————— 1s 3ms/step - loss: 5.8888e-05 - val_loss: 1.3814e-04
Epoch 10/50
406/406 ————— 1s 4ms/step - loss: 5.7702e-05 - val_loss: 1.3381e-04
Epoch 11/50
406/406 ————— 1s 3ms/step - loss: 5.6350e-05 - val_loss: 1.3035e-04
Epoch 12/50
406/406 ————— 1s 3ms/step - loss: 5.4843e-05 - val_loss: 1.2656e-04
Epoch 13/50
406/406 ————— 1s 3ms/step - loss: 5.3267e-05 - val_loss: 1.2257e-04
Epoch 14/50
406/406 ————— 1s 3ms/step - loss: 5.1690e-05 - val_loss: 1.1854e-04
Epoch 15/50
406/406 ————— 1s 3ms/step - loss: 5.0056e-05 - val_loss: 1.1481e-04
Epoch 16/50
406/406 ————— 1s 3ms/step - loss: 4.8411e-05 - val_loss: 1.1227e-04
Epoch 17/50
406/406 ————— 1s 3ms/step - loss: 4.6694e-05 - val_loss: 1.0911e-04
Epoch 18/50
406/406 ————— 1s 3ms/step - loss: 4.5015e-05 - val_loss: 1.0675e-04
Epoch 19/50
406/406 ————— 1s 3ms/step - loss: 4.3552e-05 - val_loss: 1.0386e-04
Epoch 20/50
406/406 ————— 1s 3ms/step - loss: 4.2195e-05 - val_loss: 1.0236e-04
Epoch 21/50
406/406 ————— 1s 3ms/step - loss: 4.0906e-05 - val_loss: 1.0115e-04

Epoch 22/50
406/406  1s 3ms/step - loss: 3.9857e-05 - val_loss: 9.8939e-05
Epoch 23/50
406/406  1s 3ms/step - loss: 3.8847e-05 - val_loss: 9.8311e-05
Epoch 24/50
406/406  1s 3ms/step - loss: 3.8096e-05 - val_loss: 9.6230e-05
Epoch 25/50
406/406  1s 3ms/step - loss: 3.7545e-05 - val_loss: 9.6165e-05
Epoch 26/50
406/406  1s 3ms/step - loss: 3.6942e-05 - val_loss: 9.6589e-05
Epoch 27/50
406/406  1s 3ms/step - loss: 3.6397e-05 - val_loss: 9.4553e-05
Epoch 28/50
406/406  1s 3ms/step - loss: 3.5748e-05 - val_loss: 9.0217e-05
Epoch 29/50
406/406  1s 3ms/step - loss: 3.5474e-05 - val_loss: 9.2739e-05
Epoch 30/50
406/406  1s 3ms/step - loss: 3.5230e-05 - val_loss: 9.1745e-05
Epoch 31/50
406/406  1s 3ms/step - loss: 3.4889e-05 - val_loss: 8.9335e-05
Epoch 32/50
406/406  1s 3ms/step - loss: 3.4581e-05 - val_loss: 8.8234e-05
Epoch 33/50
406/406  1s 3ms/step - loss: 3.4153e-05 - val_loss: 8.6594e-05
Epoch 34/50
406/406  1s 3ms/step - loss: 3.3948e-05 - val_loss: 8.5692e-05
Epoch 35/50
406/406  1s 3ms/step - loss: 3.3753e-05 - val_loss: 8.6247e-05
Epoch 36/50
406/406  1s 3ms/step - loss: 3.3531e-05 - val_loss: 8.4968e-05
Epoch 37/50
406/406  1s 3ms/step - loss: 3.3286e-05 - val_loss: 8.5683e-05
Epoch 38/50
406/406  1s 3ms/step - loss: 3.3411e-05 - val_loss: 8.4828e-05
Epoch 39/50
406/406  1s 3ms/step - loss: 3.3033e-05 - val_loss: 8.4599e-05
Epoch 40/50
406/406  1s 3ms/step - loss: 3.3167e-05 - val_loss: 8.4824e-05
Epoch 41/50
406/406  1s 3ms/step - loss: 3.2844e-05 - val_loss: 8.4972e-05
Epoch 42/50
406/406  1s 3ms/step - loss: 3.2769e-05 - val_loss: 8.5022e-05

Epoch 43/50
406/406 ————— 1s 3ms/step - loss: 3.2692e-05 - val_loss: 8.5204e-05
Epoch 44/50
406/406 ————— 2s 4ms/step - loss: 3.2637e-05 - val_loss: 8.5360e-05
Epoch 45/50
406/406 ————— 1s 3ms/step - loss: 3.2685e-05 - val_loss: 8.5863e-05
Epoch 46/50
406/406 ————— 1s 3ms/step - loss: 3.2445e-05 - val_loss: 8.5620e-05
Epoch 47/50
406/406 ————— 1s 3ms/step - loss: 3.2281e-05 - val_loss: 8.6822e-05
Epoch 48/50
406/406 ————— 1s 3ms/step - loss: 3.2376e-05 - val_loss: 8.5149e-05
Epoch 49/50
406/406 ————— 1s 3ms/step - loss: 3.2409e-05 - val_loss: 8.6789e-05
Epoch 50/50
406/406 ————— 1s 3ms/step - loss: 3.2289e-05 - val_loss: 8.7389e-05

In [19]: *# Plot Training and Validation Loss for IBM Baseline Model*

```
plt.figure(figsize=(8, 5))
plt.plot(history_baseline_ibm.history["loss"], label="Training Loss")
plt.plot(history_baseline_ibm.history["val_loss"], label="Validation Loss")
plt.title("IBM Baseline LSTM Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.show()
```



Berdasarkan grafik loss baseline LSTM pada saham IBM, training loss mengalami penurunan yang cukup besar pada epoch awal, kemudian mulai stabil pada nilai yang rendah. Validation loss juga menunjukkan tren penurunan secara bertahap selama proses training.

Pada hasil akhir, baseline LSTM untuk saham IBM memperoleh final training loss sebesar 0.000032 dan final validation loss sebesar 0.000087. Nilai validation loss masih lebih tinggi dibandingkan training loss, tetapi keduanya berada pada nilai yang relatif kecil. Hal ini menunjukkan bahwa baseline LSTM mampu mempelajari pola historis harga saham IBM dengan cukup baik.

Hasil baseline IBM ini digunakan sebagai pembandingan awal sebelum dilakukan modifikasi arsitektur LSTM.

In [20]: # Baseline Loss Summary

```
baseline_loss_summary = pd.DataFrame({
    "Stock": ["FB", "IBM"],
    "Final Training Loss": [
        history_baseline_fb.history["loss"][-1],
        history_baseline_ibm.history["loss"][-1]
    ],
    "Final Validation Loss": [
        history_baseline_fb.history["val_loss"][-1],
        history_baseline_ibm.history["val_loss"][-1]
    ]
})

baseline_loss_summary
```

Out[20]:

	Stock	Final Training Loss	Final Validation Loss
0	FB	0.000178	0.001165
1	IBM	0.000032	0.000087

Berdasarkan baseline loss summary, model baseline LSTM pada saham FB memperoleh final training loss sebesar 0.000178 dan final validation loss sebesar 0.001165. Validation loss yang lebih tinggi dibandingkan training loss menunjukkan adanya gap antara performa model pada train set dan validation set.

Pada saham IBM, model baseline memperoleh final training loss sebesar 0.000032 dan final validation loss sebesar 0.000087. Nilai validation loss masih lebih tinggi dibandingkan training loss, tetapi gap-nya lebih kecil dibandingkan FB. Hal ini menunjukkan bahwa baseline LSTM pada saham IBM memiliki kemampuan generalisasi yang cukup stabil pada validation set.

Secara keseluruhan, baseline LSTM berhasil dibangun dan dilatih untuk dua saham, yaitu FB dan IBM. Kedua model menggunakan arsitektur yang sama, yaitu satu layer LSTM dengan 50 units dan satu output neuron. Model baseline ini kemudian digunakan sebagai pembanding terhadap modified LSTM pada bagian berikutnya.

1c. Modified LSTM Architecture

Pada bagian ini dilakukan modifikasi terhadap arsitektur baseline LSTM untuk mencoba meningkatkan performa model. Baseline sebelumnya menggunakan satu layer LSTM dengan 50 units, activation function ReLU, dan satu output neuron. Pada model modifikasi, arsitektur dibuat lebih kompleks dengan menambahkan jumlah units, Dropout, Dense layer tambahan, learning rate yang lebih kecil, dan EarlyStopping.

Modifikasi ini dilakukan karena pada baseline model masih terdapat gap antara training loss dan validation loss, terutama pada saham FB. Dropout digunakan untuk membantu mengurangi risiko overfitting, sedangkan learning rate yang lebih kecil digunakan agar proses training lebih stabil. EarlyStopping digunakan agar training dapat berhenti ketika validation loss tidak lagi membaik.

Model modified tetap dilatih menggunakan train set dan dipantau menggunakan validation set. Test set tidak digunakan pada tahap training karena test set hanya digunakan untuk evaluasi akhir pada bagian 1d.

```
In [21]: # Additional Import for Modified LSTM
```

```
from tensorflow.keras.layers import Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

```
In [22]: # Build Modified LSTM Model
```

```
def build_modified_lstm(input_shape):
    model = Sequential()
    model.add(Input(shape=input_shape))

    model.add(LSTM(units=64, activation="relu"))
    model.add(Dropout(0.2))

    model.add(Dense(16, activation="relu"))
    model.add(Dense(1))

    model.compile(
        optimizer=Adam(learning_rate=0.0005),
        loss="mse"
    )

    return model
```

```
In [23]: # Modified LSTM for FB
```

```
modified_fb = build_modified_lstm((X_train_fb.shape[1], X_train_fb.shape[2]))  
  
modified_fb.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 64)	16,896
dropout (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 16)	1,040
dense_3 (Dense)	(None, 1)	17

Total params: 17,953 (70.13 KB)

Trainable params: 17,953 (70.13 KB)

Non-trainable params: 0 (0.00 B)

In [24]: *# Early Stopping for FB*

```
early_stop_fb = EarlyStopping(  
    monitor="val_loss",  
    patience=15,  
    restore_best_weights=True  
)
```

In [25]: *# Train Modified LSTM for FB*

```
history_modified_fb = modified_fb.fit(  
    X_train_fb,  
    y_train_fb,  
    validation_data=(X_val_fb, y_val_fb),  
    epochs=70,  
    batch_size=32,  
    callbacks=[early_stop_fb],  
    verbose=1  
)
```

```

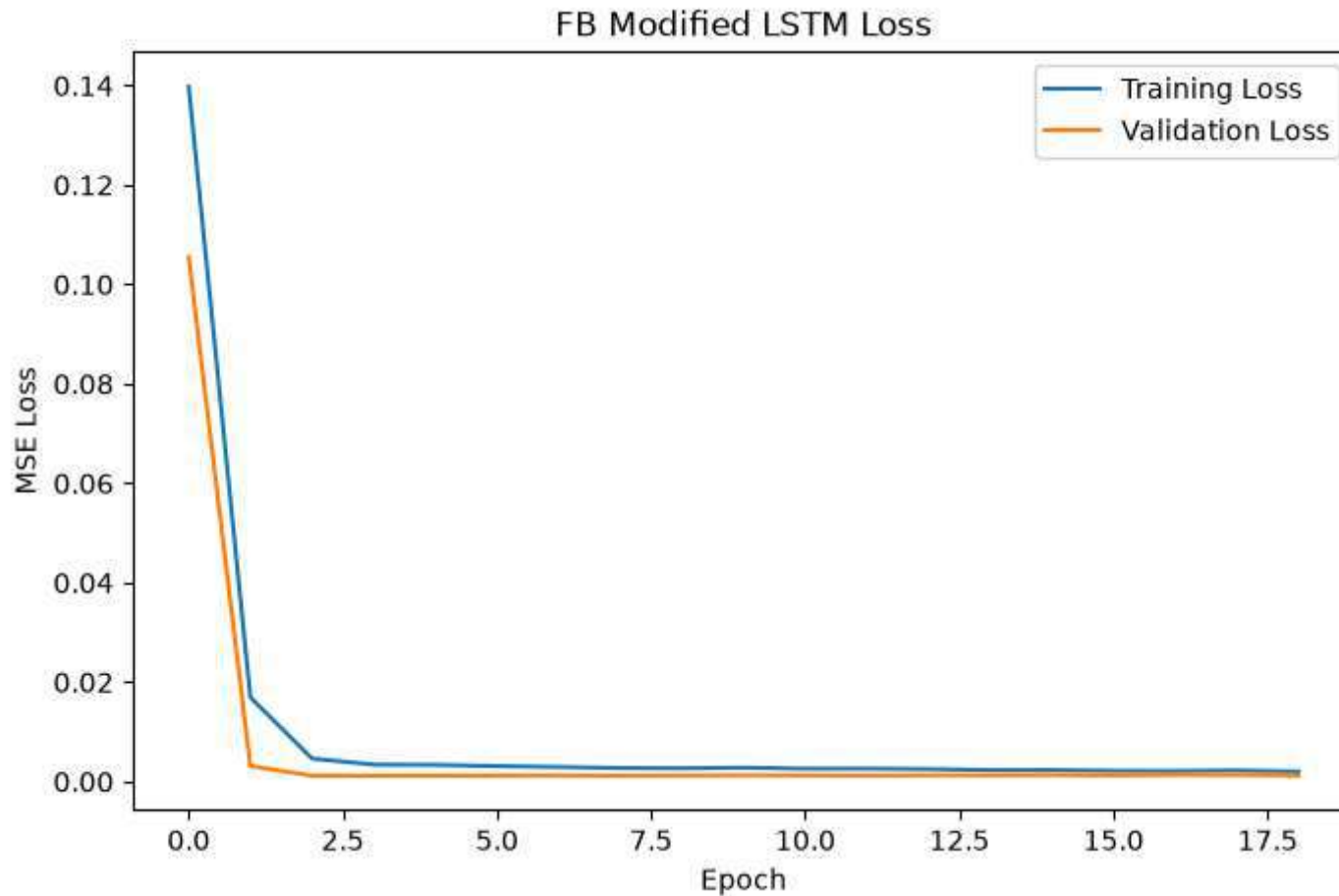
Epoch 1/70
49/49 ————— 2s 10ms/step - loss: 0.1398 - val_loss: 0.1055
Epoch 2/70
49/49 ————— 0s 4ms/step - loss: 0.0169 - val_loss: 0.0031
Epoch 3/70
49/49 ————— 0s 5ms/step - loss: 0.0046 - val_loss: 0.0012
Epoch 4/70
49/49 ————— 0s 4ms/step - loss: 0.0034 - val_loss: 0.0011
Epoch 5/70
49/49 ————— 0s 5ms/step - loss: 0.0033 - val_loss: 0.0011
Epoch 6/70
49/49 ————— 0s 5ms/step - loss: 0.0031 - val_loss: 0.0012
Epoch 7/70
49/49 ————— 0s 5ms/step - loss: 0.0029 - val_loss: 0.0012
Epoch 8/70
49/49 ————— 0s 5ms/step - loss: 0.0027 - val_loss: 0.0012
Epoch 9/70
49/49 ————— 0s 5ms/step - loss: 0.0027 - val_loss: 0.0012
Epoch 10/70
49/49 ————— 0s 5ms/step - loss: 0.0028 - val_loss: 0.0013
Epoch 11/70
49/49 ————— 0s 5ms/step - loss: 0.0026 - val_loss: 0.0012
Epoch 12/70
49/49 ————— 0s 5ms/step - loss: 0.0026 - val_loss: 0.0012
Epoch 13/70
49/49 ————— 0s 6ms/step - loss: 0.0025 - val_loss: 0.0012
Epoch 14/70
49/49 ————— 0s 5ms/step - loss: 0.0023 - val_loss: 0.0012
Epoch 15/70
49/49 ————— 0s 5ms/step - loss: 0.0023 - val_loss: 0.0013
Epoch 16/70
49/49 ————— 0s 5ms/step - loss: 0.0021 - val_loss: 0.0012
Epoch 17/70
49/49 ————— 0s 5ms/step - loss: 0.0021 - val_loss: 0.0013
Epoch 18/70
49/49 ————— 0s 5ms/step - loss: 0.0022 - val_loss: 0.0013
Epoch 19/70
49/49 ————— 0s 5ms/step - loss: 0.0020 - val_loss: 0.0012

```

In [26]: *# Plot Training and Validation Loss for FB Modified Model*

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(history_modified_fb.history["loss"], label="Training Loss")
plt.plot(history_modified_fb.history["val_loss"], label="Validation Loss")
plt.title("FB Modified LSTM Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.show()
```



Berdasarkan grafik loss modified LSTM pada saham FB, training loss dan validation loss mengalami penurunan tajam pada epoch awal. Model menggunakan EarlyStopping, sehingga training berhenti pada epoch ke-19 ketika validation loss tidak lagi menunjukkan perbaikan yang signifikan.

Pada hasil akhir, modified LSTM untuk saham FB memperoleh final training loss sebesar 0.001991 dan final validation loss sebesar 0.001213. Nilai training loss dan validation loss relatif dekat, sehingga model tidak menunjukkan gap generalisasi yang terlalu besar.

Namun, hasil modified LSTM pertama belum cukup kuat untuk dipilih sebagai model terbaik. Oleh karena itu, dilakukan tuning lanjutan dengan membangun Modified LSTM V2.

```
In [27]: # Modified LSTM for IBM

modified_ibm = build_modified_lstm((X_train_ibm.shape[1], X_train_ibm.shape[2]))

modified_ibm.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
lstm_3 (LSTM)	(None, 64)	16,896
dropout_1 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 16)	1,040
dense_5 (Dense)	(None, 1)	17

Total params: 17,953 (70.13 KB)

Trainable params: 17,953 (70.13 KB)

Non-trainable params: 0 (0.00 B)

```
In [28]: # Early Stopping Callback for IBM

early_stop_ibm = EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True
)
```

```
In [29]: # Train Modified LSTM for IBM
```

```
history_modified_ibm = modified_ibm.fit(  
    X_train_ibm,  
    y_train_ibm,  
    validation_data=(X_val_ibm, y_val_ibm),  
    epochs=70,  
    batch_size=32,  
    callbacks=[early_stop_ibm],  
    verbose=1  
)
```

Epoch 1/70
406/406  4s 4ms/step - loss: 0.0140 - val_loss: 2.4306e-04

Epoch 2/70
406/406  2s 4ms/step - loss: 0.0010 - val_loss: 5.3141e-04

Epoch 3/70
406/406  2s 4ms/step - loss: 7.5717e-04 - val_loss: 2.9253e-04

Epoch 4/70
406/406  2s 4ms/step - loss: 6.1353e-04 - val_loss: 2.0749e-04

Epoch 5/70
406/406  2s 4ms/step - loss: 5.5364e-04 - val_loss: 5.0552e-04

Epoch 6/70
406/406  2s 4ms/step - loss: 4.8287e-04 - val_loss: 4.0971e-04

Epoch 7/70
406/406  2s 4ms/step - loss: 4.3190e-04 - val_loss: 2.1472e-04

Epoch 8/70
406/406  2s 4ms/step - loss: 3.9632e-04 - val_loss: 4.6971e-04

Epoch 9/70
406/406  2s 4ms/step - loss: 3.7389e-04 - val_loss: 2.2838e-04

Epoch 10/70
406/406  2s 4ms/step - loss: 3.3736e-04 - val_loss: 3.0782e-04

Epoch 11/70
406/406  2s 4ms/step - loss: 3.2205e-04 - val_loss: 2.0530e-04

Epoch 12/70
406/406  2s 4ms/step - loss: 3.0901e-04 - val_loss: 1.9875e-04

Epoch 13/70
406/406  2s 4ms/step - loss: 3.0950e-04 - val_loss: 2.4315e-04

Epoch 14/70
406/406  1s 4ms/step - loss: 2.9044e-04 - val_loss: 1.9351e-04

Epoch 15/70
406/406  2s 4ms/step - loss: 2.8567e-04 - val_loss: 2.9538e-04

Epoch 16/70
406/406  2s 4ms/step - loss: 2.7664e-04 - val_loss: 1.9068e-04

Epoch 17/70
406/406  1s 4ms/step - loss: 2.6901e-04 - val_loss: 2.1252e-04

Epoch 18/70
406/406  2s 4ms/step - loss: 2.3425e-04 - val_loss: 5.7937e-04

Epoch 19/70
406/406  2s 5ms/step - loss: 2.3726e-04 - val_loss: 3.9677e-04

Epoch 20/70
406/406  2s 5ms/step - loss: 2.1349e-04 - val_loss: 2.0200e-04

Epoch 21/70
406/406  2s 4ms/step - loss: 2.1520e-04 - val_loss: 2.0987e-04

Epoch 22/70
406/406  2s 4ms/step - loss: 2.1311e-04 - val_loss: 1.9747e-04

Epoch 23/70
406/406  2s 4ms/step - loss: 1.9897e-04 - val_loss: 1.7604e-04

Epoch 24/70
406/406  2s 4ms/step - loss: 1.9096e-04 - val_loss: 2.3419e-04

Epoch 25/70
406/406  2s 4ms/step - loss: 1.9474e-04 - val_loss: 1.9631e-04

Epoch 26/70
406/406  2s 4ms/step - loss: 1.7936e-04 - val_loss: 1.6467e-04

Epoch 27/70
406/406  2s 4ms/step - loss: 1.7328e-04 - val_loss: 1.9799e-04

Epoch 28/70
406/406  2s 4ms/step - loss: 1.5680e-04 - val_loss: 2.3761e-04

Epoch 29/70
406/406  2s 4ms/step - loss: 1.5615e-04 - val_loss: 1.8904e-04

Epoch 30/70
406/406  2s 4ms/step - loss: 1.4636e-04 - val_loss: 2.1197e-04

Epoch 31/70
406/406  2s 4ms/step - loss: 1.4293e-04 - val_loss: 1.5570e-04

Epoch 32/70
406/406  2s 4ms/step - loss: 1.4619e-04 - val_loss: 1.6695e-04

Epoch 33/70
406/406  2s 4ms/step - loss: 1.3483e-04 - val_loss: 2.0969e-04

Epoch 34/70
406/406  2s 4ms/step - loss: 1.3554e-04 - val_loss: 1.6011e-04

Epoch 35/70
406/406  2s 4ms/step - loss: 1.2065e-04 - val_loss: 1.4996e-04

Epoch 36/70
406/406  2s 4ms/step - loss: 1.2778e-04 - val_loss: 1.5306e-04

Epoch 37/70
406/406  2s 4ms/step - loss: 1.2317e-04 - val_loss: 2.3350e-04

Epoch 38/70
406/406  2s 4ms/step - loss: 1.1526e-04 - val_loss: 2.3662e-04

Epoch 39/70
406/406  2s 4ms/step - loss: 1.1812e-04 - val_loss: 1.8698e-04

Epoch 40/70
406/406  2s 4ms/step - loss: 1.1224e-04 - val_loss: 1.5963e-04

Epoch 41/70
406/406  2s 4ms/step - loss: 1.1330e-04 - val_loss: 2.3346e-04

Epoch 42/70
406/406  2s 4ms/step - loss: 1.1434e-04 - val_loss: 1.2847e-04

```

Epoch 43/70
406/406 ————— 2s 4ms/step - loss: 1.0533e-04 - val_loss: 1.6293e-04
Epoch 44/70
406/406 ————— 2s 4ms/step - loss: 1.0549e-04 - val_loss: 1.8913e-04
Epoch 45/70
406/406 ————— 2s 4ms/step - loss: 1.0711e-04 - val_loss: 1.7430e-04
Epoch 46/70
406/406 ————— 2s 4ms/step - loss: 1.0403e-04 - val_loss: 2.4099e-04
Epoch 47/70
406/406 ————— 2s 4ms/step - loss: 9.8792e-05 - val_loss: 1.5769e-04
Epoch 48/70
406/406 ————— 2s 4ms/step - loss: 1.0246e-04 - val_loss: 1.7873e-04
Epoch 49/70
406/406 ————— 2s 4ms/step - loss: 9.6732e-05 - val_loss: 1.8426e-04
Epoch 50/70
406/406 ————— 2s 4ms/step - loss: 9.7826e-05 - val_loss: 2.5896e-04
Epoch 51/70
406/406 ————— 2s 4ms/step - loss: 9.7679e-05 - val_loss: 2.1329e-04
Epoch 52/70
406/406 ————— 2s 4ms/step - loss: 9.4924e-05 - val_loss: 1.6032e-04
Epoch 53/70
406/406 ————— 2s 4ms/step - loss: 9.5813e-05 - val_loss: 2.1404e-04
Epoch 54/70
406/406 ————— 2s 4ms/step - loss: 9.6128e-05 - val_loss: 1.5515e-04
Epoch 55/70
406/406 ————— 2s 4ms/step - loss: 9.2362e-05 - val_loss: 2.9756e-04
Epoch 56/70
406/406 ————— 2s 4ms/step - loss: 9.5074e-05 - val_loss: 1.8013e-04
Epoch 57/70
406/406 ————— 2s 4ms/step - loss: 8.5616e-05 - val_loss: 2.0925e-04

```

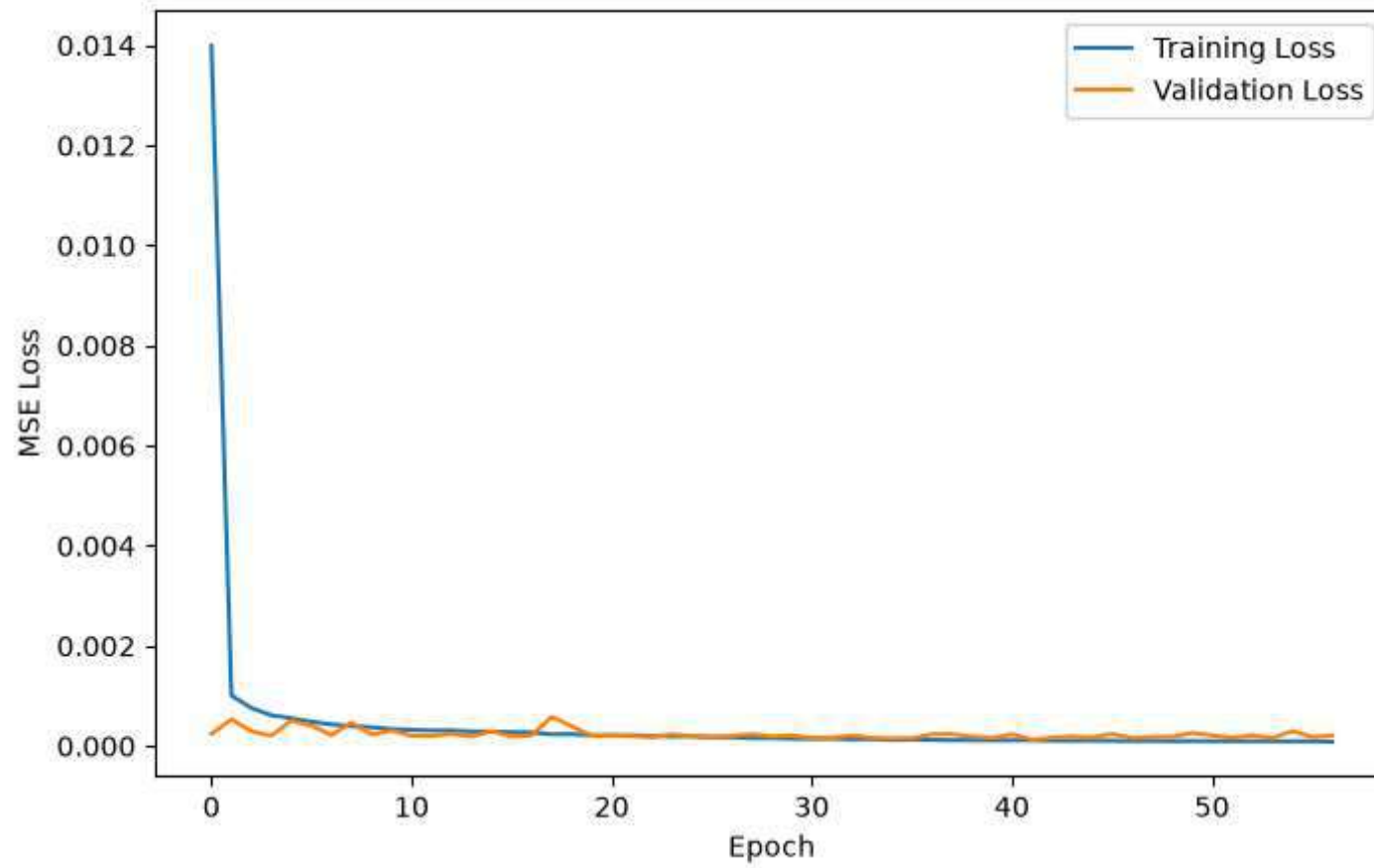
In [30]: *# Plot Training and Validation Loss for IBM Modified Model*

```

plt.figure(figsize=(8, 5))
plt.plot(history_modified_ibm.history["loss"], label="Training Loss")
plt.plot(history_modified_ibm.history["val_loss"], label="Validation Loss")
plt.title("IBM Modified LSTM Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.show()

```

IBM Modified LSTM Loss



Berdasarkan grafik loss modified LSTM pada saham IBM, training loss mengalami penurunan selama proses training. Hal ini menunjukkan bahwa model mampu mempelajari pola dari train set. Namun, validation loss terlihat lebih fluktuatif dan tidak selalu mengikuti penurunan training loss.

Model menggunakan epoch maksimum sebesar 70 dan EarlyStopping. Pada proses training, model berhenti pada epoch ke-57 karena validation loss tidak lagi menunjukkan perbaikan yang signifikan setelah beberapa epoch. Dengan `restore_best_weights=True`, model tetap menggunakan bobot terbaik berdasarkan validation loss.

Fluktuasi validation loss menunjukkan bahwa modified LSTM pada saham IBM kurang stabil dibandingkan baseline model. Oleh karena itu, model ini tetap perlu dibandingkan secara objektif dengan baseline dan Modified LSTM V2 menggunakan RMSE, MAE, dan MAPE pada test set.

In [31]: *# Modified Model Loss Summary*

```
modified_loss_summary = pd.DataFrame({
    "Stock": ["FB", "IBM"],
    "Final Training Loss": [
        history_modified_fb.history["loss"][-1],
        history_modified_ibm.history["loss"][-1]
    ],
    "Final Validation Loss": [
        history_modified_fb.history["val_loss"][-1],
        history_modified_ibm.history["val_loss"][-1]
    ],
    "Epochs Used": [
        len(history_modified_fb.history["loss"]),
        len(history_modified_ibm.history["loss"])
    ]
})

modified_loss_summary
```

Out[31]:

	Stock	Final Training Loss	Final Validation Loss	Epochs Used
0	FB	0.001991	0.001213	19
1	IBM	0.000086	0.000209	57

Berdasarkan modified loss summary, modified LSTM pada saham FB memperoleh final training loss sebesar 0.001991 dan final validation loss sebesar 0.001213. Model berhenti pada epoch ke-19 karena menggunakan EarlyStopping. Nilai training loss dan validation loss relatif masih dekat, sehingga modified model pada saham FB tidak menunjukkan gap yang terlalu besar antara train set dan validation set.

Pada saham IBM, modified LSTM memperoleh final training loss sebesar 0.000086 dan final validation loss sebesar 0.000209. Model berhenti pada epoch ke-57 karena validation loss tidak lagi membaik setelah beberapa epoch. Validation loss yang lebih tinggi dibandingkan training loss menunjukkan bahwa model modified IBM masih memiliki gap generalisasi, meskipun nilainya tidak sebesar interpretasi sebelumnya.

Secara keseluruhan, modified LSTM berhasil dibangun untuk saham FB dan IBM dengan meningkatkan jumlah LSTM units, menambahkan Dropout, menambahkan Dense layer tambahan, menggunakan learning rate yang lebih kecil, serta menerapkan EarlyStopping. Namun, hasil modified LSTM pertama belum cukup kuat untuk dipilih sebagai model terbaik, sehingga dilakukan tuning lanjutan melalui Modified LSTM V2.

```
In [32]: # Validation Evaluation Function
from sklearn.metrics import mean_squared_error, mean_absolute_error

def evaluate_lstm_on_validation(model, X_val, y_val, scaler):
    y_pred_scaled = model.predict(X_val)

    y_actual = scaler.inverse_transform(y_val.reshape(-1, 1))
    y_pred = scaler.inverse_transform(y_pred_scaled)

    rmse = np.sqrt(mean_squared_error(y_actual, y_pred))
    mae = mean_absolute_error(y_actual, y_pred)
    mape = np.mean(np.abs((y_actual - y_pred) / y_actual)) * 100

    return rmse, mae, mape
```

```
In [33]: # Baseline vs Modified LSTM Validation Evaluation

fb_baseline_val_rmse, fb_baseline_val_mae, fb_baseline_val_mape = evaluate_lstm_on_validation(
    baseline_fb, X_val_fb, y_val_fb, fb_scaler
)

fb_modified_val_rmse, fb_modified_val_mae, fb_modified_val_mape = evaluate_lstm_on_validation(
```

```

        modified_fb, X_val_fb, y_val_fb, fb_scaler
    )

    ibm_baseline_val_rmse, ibm_baseline_val_mae, ibm_baseline_val_mape = evaluate_lstm_on_validation(
        baseline_ibm, X_val_ibm, y_val_ibm, ibm_scaler
    )

    ibm_modified_val_rmse, ibm_modified_val_mae, ibm_modified_val_mape = evaluate_lstm_on_validation(
        modified_ibm, X_val_ibm, y_val_ibm, ibm_scaler
    )

    baseline_vs_modified_result = pd.DataFrame({
        "Stock": ["FB", "FB", "IBM", "IBM"],
        "Model": ["Baseline LSTM", "Modified LSTM", "Baseline LSTM", "Modified LSTM"],
        "RMSE": [
            fb_baseline_val_rmse,
            fb_modified_val_rmse,
            ibm_baseline_val_rmse,
            ibm_modified_val_rmse
        ],
        "MAE": [
            fb_baseline_val_mae,
            fb_modified_val_mae,
            ibm_baseline_val_mae,
            ibm_modified_val_mae
        ],
        "MAPE (%)": [
            fb_baseline_val_mape,
            fb_modified_val_mape,
            ibm_baseline_val_mape,
            ibm_modified_val_mape
        ]
    })

    baseline_vs_modified_result

```

6/6  0s 35ms/step

6/6  0s 38ms/step

WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x000001BD688EA0C0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

46/46  0s 5ms/step

46/46  0s 5ms/step

Out[33]:

	Stock	Model	RMSE	MAE	MAPE (%)
0	FB	Baseline LSTM	6.561306	4.192379	2.657850
1	FB	Modified LSTM	6.501971	4.212396	2.657592
2	IBM	Baseline LSTM	1.979208	1.389167	0.890948
3	IBM	Modified LSTM	2.399733	1.759683	1.124649

Baseline vs Modified LSTM Comparison

Berdasarkan tabel perbandingan pada validation set, hasil modified LSTM pertama belum sepenuhnya konsisten dibandingkan baseline. Pada saham FB, modified LSTM pertama menghasilkan RMSE dan MAPE yang sedikit lebih rendah dibandingkan baseline, tetapi MAE baseline masih sedikit lebih baik. Hal ini menunjukkan bahwa peningkatan pada modified model pertama belum terlalu signifikan.

Pada saham IBM, baseline LSTM masih memberikan hasil yang lebih baik dibandingkan modified LSTM pertama. Hal ini terlihat dari nilai RMSE, MAE, dan MAPE baseline yang lebih rendah dibandingkan modified model.

Berdasarkan hasil tersebut, modified LSTM pertama belum cukup kuat untuk dijadikan model terbaik. Oleh karena itu, dilakukan tuning lanjutan dengan membangun Modified LSTM V2 agar performa model dapat meningkat pada validation set maupun test set.

Modified LSTM V2

Karena modified LSTM pertama belum berhasil mengungguli baseline, maka dibuat Modified LSTM V2. Modified LSTM V2 dibuat sebagai bentuk hyperparameter tuning lanjutan. Karena saham FB dan IBM memiliki jumlah data dan pola historis yang berbeda, konfigurasi V2 disesuaikan untuk masing-masing saham. Tujuannya adalah mencari konfigurasi yang lebih stabil dan menghasilkan error yang lebih rendah pada validation dan test set.

In [34]: *# Build Modified LSTM V2 Model*

```
def build_modified_lstm_v2(input_shape):
    model = Sequential()
    model.add(Input(shape=input_shape))

    model.add(LSTM(units=64, activation="relu"))
    model.add(Dense(8, activation="relu"))
    model.add(Dense(1))

    model.compile(
        optimizer=Adam(learning_rate=0.0003),
        loss="mse"
    )

    return model
```

In [35]: *# Modified LSTM V2 for FB*

```
modified_v2_fb = build_modified_lstm_v2((X_train_fb.shape[1], X_train_fb.shape[2]))
modified_v2_fb.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
lstm_4 (LSTM)	(None, 64)	16,896
dense_6 (Dense)	(None, 8)	520
dense_7 (Dense)	(None, 1)	9

Total params: 17,425 (68.07 KB)

Trainable params: 17,425 (68.07 KB)

Non-trainable params: 0 (0.00 B)

In [36]: *# Early Stopping for FB Modified V2*

```
early_stop_v2_fb = EarlyStopping(  
    monitor="val_loss",  
    patience=30,  
    min_delta=1e-7,  
    restore_best_weights=True  
)
```

In [37]: *# Train Modified LSTM V2 for FB*

```
modified_v2_fb = build_modified_lstm_v2((X_train_fb.shape[1], X_train_fb.shape[2]))  
  
history_modified_v2_fb = modified_v2_fb.fit(  
    X_train_fb,  
    y_train_fb,  
    validation_data=(X_val_fb, y_val_fb),  
    epochs=150,  
    batch_size=16,  
    callbacks=[early_stop_v2_fb],  
    verbose=1  
)
```

Epoch 1/150
97/97  3s 6ms/step - loss: 0.1314 - val_loss: 0.1028
Epoch 2/150
97/97  0s 4ms/step - loss: 0.0078 - val_loss: 0.0011
Epoch 3/150
97/97  0s 4ms/step - loss: 1.8302e-04 - val_loss: 0.0012
Epoch 4/150
97/97  0s 4ms/step - loss: 1.7694e-04 - val_loss: 0.0012
Epoch 5/150
97/97  0s 4ms/step - loss: 1.7391e-04 - val_loss: 0.0012
Epoch 6/150
97/97  0s 4ms/step - loss: 1.7250e-04 - val_loss: 0.0011
Epoch 7/150
97/97  0s 4ms/step - loss: 1.7184e-04 - val_loss: 0.0011
Epoch 8/150
97/97  0s 4ms/step - loss: 1.7058e-04 - val_loss: 0.0011
Epoch 9/150
97/97  0s 4ms/step - loss: 1.6973e-04 - val_loss: 0.0011
Epoch 10/150
97/97  0s 4ms/step - loss: 1.6962e-04 - val_loss: 0.0011
Epoch 11/150
97/97  0s 4ms/step - loss: 1.6901e-04 - val_loss: 0.0011
Epoch 12/150
97/97  0s 4ms/step - loss: 1.6858e-04 - val_loss: 0.0011
Epoch 13/150
97/97  0s 4ms/step - loss: 1.6831e-04 - val_loss: 0.0011
Epoch 14/150
97/97  0s 4ms/step - loss: 1.6795e-04 - val_loss: 0.0011
Epoch 15/150
97/97  0s 4ms/step - loss: 1.6762e-04 - val_loss: 0.0011
Epoch 16/150
97/97  0s 4ms/step - loss: 1.6726e-04 - val_loss: 0.0011
Epoch 17/150
97/97  0s 4ms/step - loss: 1.6681e-04 - val_loss: 0.0011
Epoch 18/150
97/97  0s 4ms/step - loss: 1.6673e-04 - val_loss: 0.0011
Epoch 19/150
97/97  0s 4ms/step - loss: 1.6640e-04 - val_loss: 0.0011
Epoch 20/150
97/97  0s 4ms/step - loss: 1.6593e-04 - val_loss: 0.0011
Epoch 21/150
97/97  0s 4ms/step - loss: 1.6581e-04 - val_loss: 0.0011

```
Epoch 22/150
97/97  0s 4ms/step - loss: 1.6548e-04 - val_loss: 0.0011
Epoch 23/150
97/97  0s 4ms/step - loss: 1.6498e-04 - val_loss: 0.0011
Epoch 24/150
97/97  0s 4ms/step - loss: 1.6479e-04 - val_loss: 0.0010
Epoch 25/150
97/97  0s 4ms/step - loss: 1.6448e-04 - val_loss: 0.0010
Epoch 26/150
97/97  0s 4ms/step - loss: 1.6411e-04 - val_loss: 0.0010
Epoch 27/150
97/97  0s 4ms/step - loss: 1.6379e-04 - val_loss: 0.0010
Epoch 28/150
97/97  0s 4ms/step - loss: 1.6341e-04 - val_loss: 0.0010
Epoch 29/150
97/97  0s 4ms/step - loss: 1.6301e-04 - val_loss: 0.0010
Epoch 30/150
97/97  0s 4ms/step - loss: 1.6271e-04 - val_loss: 0.0010
Epoch 31/150
97/97  0s 4ms/step - loss: 1.6236e-04 - val_loss: 0.0010
Epoch 32/150
97/97  0s 4ms/step - loss: 1.6196e-04 - val_loss: 0.0010
Epoch 33/150
97/97  0s 4ms/step - loss: 1.6146e-04 - val_loss: 9.9889e-04
Epoch 34/150
97/97  0s 4ms/step - loss: 1.6123e-04 - val_loss: 9.8982e-04
Epoch 35/150
97/97  0s 4ms/step - loss: 1.6065e-04 - val_loss: 9.8620e-04
Epoch 36/150
97/97  0s 4ms/step - loss: 1.6025e-04 - val_loss: 9.7923e-04
Epoch 37/150
97/97  0s 4ms/step - loss: 1.5972e-04 - val_loss: 9.7193e-04
Epoch 38/150
97/97  0s 4ms/step - loss: 1.5916e-04 - val_loss: 9.6427e-04
Epoch 39/150
97/97  0s 4ms/step - loss: 1.5931e-04 - val_loss: 9.5071e-04
Epoch 40/150
97/97  0s 4ms/step - loss: 1.5851e-04 - val_loss: 9.4408e-04
Epoch 41/150
97/97  0s 4ms/step - loss: 1.5790e-04 - val_loss: 9.3788e-04
Epoch 42/150
97/97  0s 4ms/step - loss: 1.5687e-04 - val_loss: 9.3954e-04
```

Epoch 43/150
97/97  0s 4ms/step - loss: 1.5676e-04 - val_loss: 9.2841e-04
Epoch 44/150
97/97  0s 4ms/step - loss: 1.5593e-04 - val_loss: 9.2068e-04
Epoch 45/150
97/97  0s 4ms/step - loss: 1.5504e-04 - val_loss: 9.2010e-04
Epoch 46/150
97/97  0s 4ms/step - loss: 1.5471e-04 - val_loss: 9.0933e-04
Epoch 47/150
97/97  0s 4ms/step - loss: 1.5388e-04 - val_loss: 9.0248e-04
Epoch 48/150
97/97  0s 4ms/step - loss: 1.5326e-04 - val_loss: 8.9485e-04
Epoch 49/150
97/97  0s 4ms/step - loss: 1.5267e-04 - val_loss: 8.9278e-04
Epoch 50/150
97/97  0s 4ms/step - loss: 1.5196e-04 - val_loss: 8.8613e-04
Epoch 51/150
97/97  0s 4ms/step - loss: 1.5116e-04 - val_loss: 8.7881e-04
Epoch 52/150
97/97  0s 4ms/step - loss: 1.5050e-04 - val_loss: 8.7278e-04
Epoch 53/150
97/97  0s 4ms/step - loss: 1.4986e-04 - val_loss: 8.6660e-04
Epoch 54/150
97/97  0s 4ms/step - loss: 1.4930e-04 - val_loss: 8.6406e-04
Epoch 55/150
97/97  0s 4ms/step - loss: 1.4832e-04 - val_loss: 8.5834e-04
Epoch 56/150
97/97  0s 4ms/step - loss: 1.4774e-04 - val_loss: 8.5159e-04
Epoch 57/150
97/97  0s 4ms/step - loss: 1.4687e-04 - val_loss: 8.4633e-04
Epoch 58/150
97/97  0s 4ms/step - loss: 1.4616e-04 - val_loss: 8.3844e-04
Epoch 59/150
97/97  0s 4ms/step - loss: 1.4555e-04 - val_loss: 8.3451e-04
Epoch 60/150
97/97  0s 4ms/step - loss: 1.4490e-04 - val_loss: 8.2979e-04
Epoch 61/150
97/97  0s 4ms/step - loss: 1.4433e-04 - val_loss: 8.2626e-04
Epoch 62/150
97/97  0s 4ms/step - loss: 1.4349e-04 - val_loss: 8.2235e-04
Epoch 63/150
97/97  0s 4ms/step - loss: 1.4262e-04 - val_loss: 8.1469e-04

Epoch 64/150
97/97  0s 4ms/step - loss: 1.4205e-04 - val_loss: 8.1314e-04
Epoch 65/150
97/97  0s 4ms/step - loss: 1.4126e-04 - val_loss: 8.0614e-04
Epoch 66/150
97/97  0s 5ms/step - loss: 1.4074e-04 - val_loss: 8.0351e-04
Epoch 67/150
97/97  1s 6ms/step - loss: 1.3997e-04 - val_loss: 7.9611e-04
Epoch 68/150
97/97  0s 4ms/step - loss: 1.3944e-04 - val_loss: 7.9560e-04
Epoch 69/150
97/97  0s 4ms/step - loss: 1.3871e-04 - val_loss: 7.8821e-04
Epoch 70/150
97/97  0s 4ms/step - loss: 1.3816e-04 - val_loss: 7.8599e-04
Epoch 71/150
97/97  0s 4ms/step - loss: 1.3746e-04 - val_loss: 7.7974e-04
Epoch 72/150
97/97  0s 4ms/step - loss: 1.3690e-04 - val_loss: 7.7781e-04
Epoch 73/150
97/97  0s 4ms/step - loss: 1.3622e-04 - val_loss: 7.7095e-04
Epoch 74/150
97/97  0s 4ms/step - loss: 1.3567e-04 - val_loss: 7.7052e-04
Epoch 75/150
97/97  0s 4ms/step - loss: 1.3496e-04 - val_loss: 7.6298e-04
Epoch 76/150
97/97  0s 4ms/step - loss: 1.3452e-04 - val_loss: 7.6249e-04
Epoch 77/150
97/97  0s 4ms/step - loss: 1.3380e-04 - val_loss: 7.5560e-04
Epoch 78/150
97/97  0s 4ms/step - loss: 1.3331e-04 - val_loss: 7.5534e-04
Epoch 79/150
97/97  0s 4ms/step - loss: 1.3263e-04 - val_loss: 7.4897e-04
Epoch 80/150
97/97  0s 4ms/step - loss: 1.3213e-04 - val_loss: 7.4819e-04
Epoch 81/150
97/97  0s 4ms/step - loss: 1.3149e-04 - val_loss: 7.4117e-04
Epoch 82/150
97/97  0s 4ms/step - loss: 1.3097e-04 - val_loss: 7.3546e-04
Epoch 83/150
97/97  0s 4ms/step - loss: 1.3053e-04 - val_loss: 7.3649e-04
Epoch 84/150
97/97  0s 4ms/step - loss: 1.2985e-04 - val_loss: 7.2985e-04

Epoch 85/150
97/97 ————— 0s 4ms/step - loss: 1.2939e-04 - val_loss: 7.3018e-04
Epoch 86/150
97/97 ————— 0s 4ms/step - loss: 1.2884e-04 - val_loss: 7.2444e-04
Epoch 87/150
97/97 ————— 0s 4ms/step - loss: 1.2837e-04 - val_loss: 7.2522e-04
Epoch 88/150
97/97 ————— 0s 4ms/step - loss: 1.2781e-04 - val_loss: 7.2021e-04
Epoch 89/150
97/97 ————— 0s 4ms/step - loss: 1.2735e-04 - val_loss: 7.1352e-04
Epoch 90/150
97/97 ————— 0s 4ms/step - loss: 1.2689e-04 - val_loss: 7.1450e-04
Epoch 91/150
97/97 ————— 0s 4ms/step - loss: 1.2635e-04 - val_loss: 7.0794e-04
Epoch 92/150
97/97 ————— 0s 4ms/step - loss: 1.2602e-04 - val_loss: 7.0788e-04
Epoch 93/150
97/97 ————— 0s 4ms/step - loss: 1.2541e-04 - val_loss: 7.0343e-04
Epoch 94/150
97/97 ————— 0s 4ms/step - loss: 1.2496e-04 - val_loss: 7.0234e-04
Epoch 95/150
97/97 ————— 0s 5ms/step - loss: 1.2446e-04 - val_loss: 6.9844e-04
Epoch 96/150
97/97 ————— 0s 4ms/step - loss: 1.2414e-04 - val_loss: 6.9520e-04
Epoch 97/150
97/97 ————— 0s 4ms/step - loss: 1.2368e-04 - val_loss: 6.9351e-04
Epoch 98/150
97/97 ————— 0s 4ms/step - loss: 1.2305e-04 - val_loss: 6.9075e-04
Epoch 99/150
97/97 ————— 0s 4ms/step - loss: 1.2267e-04 - val_loss: 6.8610e-04
Epoch 100/150
97/97 ————— 0s 4ms/step - loss: 1.2220e-04 - val_loss: 6.8667e-04
Epoch 101/150
97/97 ————— 0s 4ms/step - loss: 1.2169e-04 - val_loss: 6.8421e-04
Epoch 102/150
97/97 ————— 0s 4ms/step - loss: 1.2135e-04 - val_loss: 6.8341e-04
Epoch 103/150
97/97 ————— 0s 4ms/step - loss: 1.2096e-04 - val_loss: 6.7929e-04
Epoch 104/150
97/97 ————— 0s 4ms/step - loss: 1.2053e-04 - val_loss: 6.7531e-04
Epoch 105/150
97/97 ————— 0s 4ms/step - loss: 1.2029e-04 - val_loss: 6.7586e-04

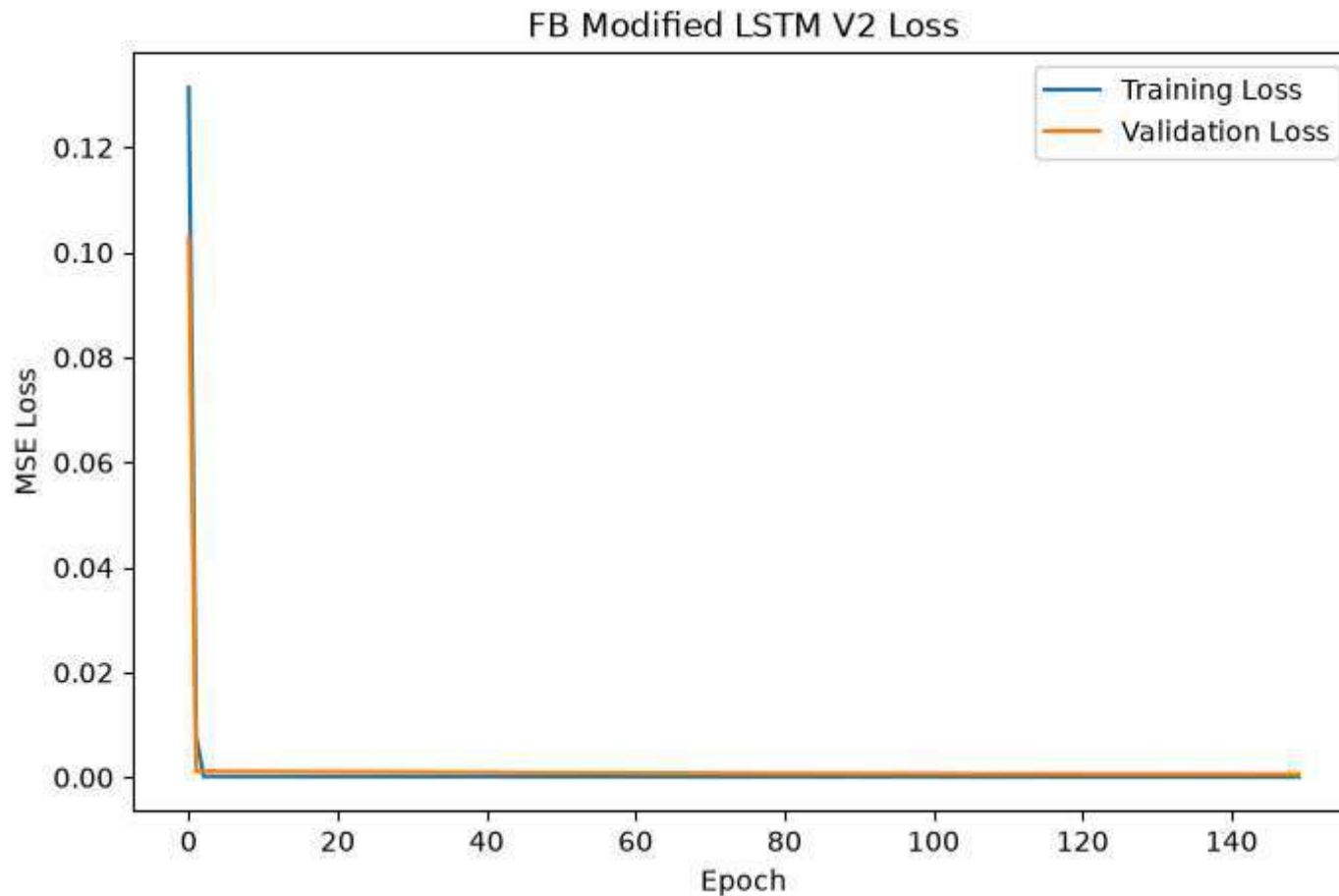
Epoch 106/150
97/97  0s 4ms/step - loss: 1.1978e-04 - val_loss: 6.7179e-04
Epoch 107/150
97/97  0s 4ms/step - loss: 1.1945e-04 - val_loss: 6.7261e-04
Epoch 108/150
97/97  0s 4ms/step - loss: 1.1907e-04 - val_loss: 6.7133e-04
Epoch 109/150
97/97  1s 5ms/step - loss: 1.1873e-04 - val_loss: 6.6189e-04
Epoch 110/150
97/97  1s 6ms/step - loss: 1.1870e-04 - val_loss: 6.6176e-04
Epoch 111/150
97/97  0s 5ms/step - loss: 1.1844e-04 - val_loss: 6.5931e-04
Epoch 112/150
97/97  1s 5ms/step - loss: 1.1809e-04 - val_loss: 6.5752e-04
Epoch 113/150
97/97  1s 6ms/step - loss: 1.1779e-04 - val_loss: 6.5743e-04
Epoch 114/150
97/97  1s 5ms/step - loss: 1.1748e-04 - val_loss: 6.5351e-04
Epoch 115/150
97/97  1s 5ms/step - loss: 1.1691e-04 - val_loss: 6.5255e-04
Epoch 116/150
97/97  1s 5ms/step - loss: 1.1661e-04 - val_loss: 6.5706e-04
Epoch 117/150
97/97  1s 5ms/step - loss: 1.1622e-04 - val_loss: 6.5493e-04
Epoch 118/150
97/97  0s 4ms/step - loss: 1.1609e-04 - val_loss: 6.5606e-04
Epoch 119/150
97/97  0s 4ms/step - loss: 1.1565e-04 - val_loss: 6.5427e-04
Epoch 120/150
97/97  0s 4ms/step - loss: 1.1549e-04 - val_loss: 6.4983e-04
Epoch 121/150
97/97  0s 4ms/step - loss: 1.1512e-04 - val_loss: 6.5205e-04
Epoch 122/150
97/97  0s 4ms/step - loss: 1.1483e-04 - val_loss: 6.4979e-04
Epoch 123/150
97/97  0s 4ms/step - loss: 1.1467e-04 - val_loss: 6.4248e-04
Epoch 124/150
97/97  0s 4ms/step - loss: 1.1458e-04 - val_loss: 6.4645e-04
Epoch 125/150
97/97  0s 4ms/step - loss: 1.1415e-04 - val_loss: 6.4036e-04
Epoch 126/150
97/97  0s 4ms/step - loss: 1.1391e-04 - val_loss: 6.3854e-04

```
Epoch 127/150
97/97  0s 4ms/step - loss: 1.1393e-04 - val_loss: 6.4176e-04
Epoch 128/150
97/97  0s 4ms/step - loss: 1.1348e-04 - val_loss: 6.4028e-04
Epoch 129/150
97/97  0s 4ms/step - loss: 1.1330e-04 - val_loss: 6.3688e-04
Epoch 130/150
97/97  0s 4ms/step - loss: 1.1272e-04 - val_loss: 6.3888e-04
Epoch 131/150
97/97  0s 4ms/step - loss: 1.1292e-04 - val_loss: 6.3644e-04
Epoch 132/150
97/97  0s 4ms/step - loss: 1.1255e-04 - val_loss: 6.3658e-04
Epoch 133/150
97/97  0s 4ms/step - loss: 1.1241e-04 - val_loss: 6.3610e-04
Epoch 134/150
97/97  0s 4ms/step - loss: 1.1215e-04 - val_loss: 6.3413e-04
Epoch 135/150
97/97  0s 4ms/step - loss: 1.1180e-04 - val_loss: 6.3546e-04
Epoch 136/150
97/97  0s 4ms/step - loss: 1.1188e-04 - val_loss: 6.3118e-04
Epoch 137/150
97/97  0s 4ms/step - loss: 1.1147e-04 - val_loss: 6.3193e-04
Epoch 138/150
97/97  0s 5ms/step - loss: 1.1215e-04 - val_loss: 6.3367e-04
Epoch 139/150
97/97  0s 4ms/step - loss: 1.1093e-04 - val_loss: 6.2891e-04
Epoch 140/150
97/97  0s 4ms/step - loss: 1.1119e-04 - val_loss: 6.2796e-04
Epoch 141/150
97/97  0s 4ms/step - loss: 1.1083e-04 - val_loss: 6.3036e-04
Epoch 142/150
97/97  0s 4ms/step - loss: 1.1151e-04 - val_loss: 6.2612e-04
Epoch 143/150
97/97  0s 4ms/step - loss: 1.1070e-04 - val_loss: 6.2490e-04
Epoch 144/150
97/97  0s 4ms/step - loss: 1.1038e-04 - val_loss: 6.2482e-04
Epoch 145/150
97/97  0s 4ms/step - loss: 1.1026e-04 - val_loss: 6.2450e-04
Epoch 146/150
97/97  0s 4ms/step - loss: 1.1004e-04 - val_loss: 6.2126e-04
Epoch 147/150
97/97  0s 4ms/step - loss: 1.0967e-04 - val_loss: 6.2264e-04
```

```
Epoch 148/150
97/97 ————— 0s 4ms/step - loss: 1.0959e-04 - val_loss: 6.2290e-04
Epoch 149/150
97/97 ————— 0s 4ms/step - loss: 1.0965e-04 - val_loss: 6.2199e-04
Epoch 150/150
97/97 ————— 0s 4ms/step - loss: 1.0928e-04 - val_loss: 6.2119e-04
```

```
In [38]: # Plot FB Modified V2 Loss
```

```
plt.figure(figsize=(8, 5))
plt.plot(history_modified_v2_fb.history["loss"], label="Training Loss")
plt.plot(history_modified_v2_fb.history["val_loss"], label="Validation Loss")
plt.title("FB Modified LSTM V2 Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.show()
```



Berdasarkan grafik loss Modified LSTM V2 pada saham FB, training loss dan validation loss mengalami penurunan yang sangat tajam pada epoch awal. Setelah beberapa epoch, kedua loss mulai stabil pada nilai yang jauh lebih rendah.

Pada hasil akhir, Modified LSTM V2 memperoleh final training loss sebesar 0.000109 dan final validation loss sebesar 0.000621. Nilai validation loss ini lebih rendah dibandingkan baseline FB dan modified LSTM pertama, sehingga Modified LSTM V2 menunjukkan peningkatan performa pada validation set.

Meskipun masih terdapat gap antara training loss dan validation loss, model ini tetap memberikan hasil terbaik pada evaluasi test set. Oleh karena itu, Modified LSTM V2 digunakan sebagai model hasil tuning untuk saham FB.

```
In [39]: def build_modified_lstm_v2_ibm(input_shape):
model = Sequential()
model.add(Input(shape=input_shape))

model.add(LSTM(units=40, activation="relu"))
model.add(Dense(1))

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="mse"
)

return model
```

```
In [40]: # Modified LSTM V2 for IBM

modified_v2_ibm = build_modified_lstm_v2_ibm((X_train_ibm.shape[1], X_train_ibm.shape[2]))
modified_v2_ibm.summary()
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
lstm_6 (LSTM)	(None, 40)	6,720
dense_10 (Dense)	(None, 1)	41

Total params: 6,761 (26.41 KB)

Trainable params: 6,761 (26.41 KB)

Non-trainable params: 0 (0.00 B)

```
In [41]: # Early Stopping for IBM Modified V2

early_stop_v2_ibm = EarlyStopping(
    monitor="val_loss",
    patience=20,
    min_delta=1e-8,
    restore_best_weights=True
)
```

In [42]: *# Train Modified LSTM V2 for IBM*

```
modified_v2_ibm = build_modified_lstm_v2_ibm((X_train_ibm.shape[1], X_train_ibm.shape[2]))

history_modified_v2_ibm = modified_v2_ibm.fit(
    X_train_ibm,
    y_train_ibm,
    validation_data=(X_val_ibm, y_val_ibm),
    epochs=120,
    batch_size=32,
    callbacks=[early_stop_v2_ibm],
    verbose=1
)
```

Epoch 1/120
406/406  3s 4ms/step - loss: 0.0069 - val_loss: 1.9213e-04

Epoch 2/120
406/406  1s 3ms/step - loss: 6.1134e-05 - val_loss: 1.6743e-04

Epoch 3/120
406/406  1s 3ms/step - loss: 5.9399e-05 - val_loss: 1.6451e-04

Epoch 4/120
406/406  1s 3ms/step - loss: 5.9690e-05 - val_loss: 1.6437e-04

Epoch 5/120
406/406  1s 4ms/step - loss: 6.0040e-05 - val_loss: 1.6428e-04

Epoch 6/120
406/406  1s 3ms/step - loss: 6.0111e-05 - val_loss: 1.6347e-04

Epoch 7/120
406/406  1s 3ms/step - loss: 5.9942e-05 - val_loss: 1.6098e-04

Epoch 8/120
406/406  1s 3ms/step - loss: 5.9575e-05 - val_loss: 1.5841e-04

Epoch 9/120
406/406  1s 3ms/step - loss: 5.9173e-05 - val_loss: 1.5486e-04

Epoch 10/120
406/406  2s 4ms/step - loss: 5.8381e-05 - val_loss: 1.5300e-04

Epoch 11/120
406/406  1s 3ms/step - loss: 5.7514e-05 - val_loss: 1.4968e-04

Epoch 12/120
406/406  1s 3ms/step - loss: 5.6483e-05 - val_loss: 1.4694e-04

Epoch 13/120
406/406  1s 3ms/step - loss: 5.5478e-05 - val_loss: 1.4251e-04

Epoch 14/120
406/406  1s 3ms/step - loss: 5.4439e-05 - val_loss: 1.4046e-04

Epoch 15/120
406/406  1s 3ms/step - loss: 5.3301e-05 - val_loss: 1.3845e-04

Epoch 16/120
406/406  1s 3ms/step - loss: 5.2154e-05 - val_loss: 1.3652e-04

Epoch 17/120
406/406  1s 3ms/step - loss: 5.1052e-05 - val_loss: 1.3496e-04

Epoch 18/120
406/406  1s 3ms/step - loss: 4.9988e-05 - val_loss: 1.3122e-04

Epoch 19/120
406/406  1s 3ms/step - loss: 4.8984e-05 - val_loss: 1.3089e-04

Epoch 20/120
406/406  2s 4ms/step - loss: 4.8002e-05 - val_loss: 1.2844e-04

Epoch 21/120
406/406  2s 4ms/step - loss: 4.6941e-05 - val_loss: 1.2596e-04

Epoch 22/120
406/406  1s 4ms/step - loss: 4.5962e-05 - val_loss: 1.2437e-04
Epoch 23/120
406/406  1s 3ms/step - loss: 4.5072e-05 - val_loss: 1.2316e-04
Epoch 24/120
406/406  1s 3ms/step - loss: 4.4101e-05 - val_loss: 1.2095e-04
Epoch 25/120
406/406  1s 3ms/step - loss: 4.3286e-05 - val_loss: 1.2007e-04
Epoch 26/120
406/406  1s 3ms/step - loss: 4.2560e-05 - val_loss: 1.1969e-04
Epoch 27/120
406/406  1s 3ms/step - loss: 4.1903e-05 - val_loss: 1.1731e-04
Epoch 28/120
406/406  1s 3ms/step - loss: 4.1088e-05 - val_loss: 1.1590e-04
Epoch 29/120
406/406  1s 3ms/step - loss: 4.0446e-05 - val_loss: 1.1361e-04
Epoch 30/120
406/406  1s 3ms/step - loss: 3.9893e-05 - val_loss: 1.1302e-04
Epoch 31/120
406/406  1s 3ms/step - loss: 3.9248e-05 - val_loss: 1.1314e-04
Epoch 32/120
406/406  1s 3ms/step - loss: 3.8725e-05 - val_loss: 1.1170e-04
Epoch 33/120
406/406  1s 3ms/step - loss: 3.8272e-05 - val_loss: 1.1069e-04
Epoch 34/120
406/406  1s 3ms/step - loss: 3.7863e-05 - val_loss: 1.0992e-04
Epoch 35/120
406/406  1s 3ms/step - loss: 3.7439e-05 - val_loss: 1.0957e-04
Epoch 36/120
406/406  1s 3ms/step - loss: 3.7088e-05 - val_loss: 1.0862e-04
Epoch 37/120
406/406  1s 3ms/step - loss: 3.6739e-05 - val_loss: 1.0638e-04
Epoch 38/120
406/406  1s 4ms/step - loss: 3.6421e-05 - val_loss: 1.0330e-04
Epoch 39/120
406/406  1s 3ms/step - loss: 3.6035e-05 - val_loss: 1.0064e-04
Epoch 40/120
406/406  1s 3ms/step - loss: 3.5708e-05 - val_loss: 9.8886e-05
Epoch 41/120
406/406  1s 3ms/step - loss: 3.5321e-05 - val_loss: 9.7103e-05
Epoch 42/120
406/406  1s 3ms/step - loss: 3.5117e-05 - val_loss: 9.7294e-05

Epoch 43/120
406/406  1s 3ms/step - loss: 3.4947e-05 - val_loss: 9.6484e-05
Epoch 44/120
406/406  1s 3ms/step - loss: 3.4756e-05 - val_loss: 9.5928e-05
Epoch 45/120
406/406  1s 3ms/step - loss: 3.4566e-05 - val_loss: 9.4877e-05
Epoch 46/120
406/406  1s 3ms/step - loss: 3.4401e-05 - val_loss: 9.3902e-05
Epoch 47/120
406/406  1s 3ms/step - loss: 3.4170e-05 - val_loss: 9.1204e-05
Epoch 48/120
406/406  1s 3ms/step - loss: 3.3960e-05 - val_loss: 9.0461e-05
Epoch 49/120
406/406  1s 3ms/step - loss: 3.3785e-05 - val_loss: 8.9907e-05
Epoch 50/120
406/406  1s 3ms/step - loss: 3.3638e-05 - val_loss: 8.9696e-05
Epoch 51/120
406/406  1s 3ms/step - loss: 3.3482e-05 - val_loss: 8.8620e-05
Epoch 52/120
406/406  1s 3ms/step - loss: 3.3333e-05 - val_loss: 8.7908e-05
Epoch 53/120
406/406  1s 3ms/step - loss: 3.3144e-05 - val_loss: 8.6971e-05
Epoch 54/120
406/406  1s 3ms/step - loss: 3.3005e-05 - val_loss: 8.6908e-05
Epoch 55/120
406/406  1s 3ms/step - loss: 3.2925e-05 - val_loss: 8.6709e-05
Epoch 56/120
406/406  1s 3ms/step - loss: 3.2800e-05 - val_loss: 8.6409e-05
Epoch 57/120
406/406  1s 3ms/step - loss: 3.2680e-05 - val_loss: 8.6227e-05
Epoch 58/120
406/406  1s 3ms/step - loss: 3.2593e-05 - val_loss: 8.5949e-05
Epoch 59/120
406/406  1s 3ms/step - loss: 3.2496e-05 - val_loss: 8.5834e-05
Epoch 60/120
406/406  1s 3ms/step - loss: 3.2424e-05 - val_loss: 8.5471e-05
Epoch 61/120
406/406  1s 3ms/step - loss: 3.2314e-05 - val_loss: 8.5178e-05
Epoch 62/120
406/406  1s 3ms/step - loss: 3.2258e-05 - val_loss: 8.5068e-05
Epoch 63/120
406/406  1s 3ms/step - loss: 3.2193e-05 - val_loss: 8.4969e-05

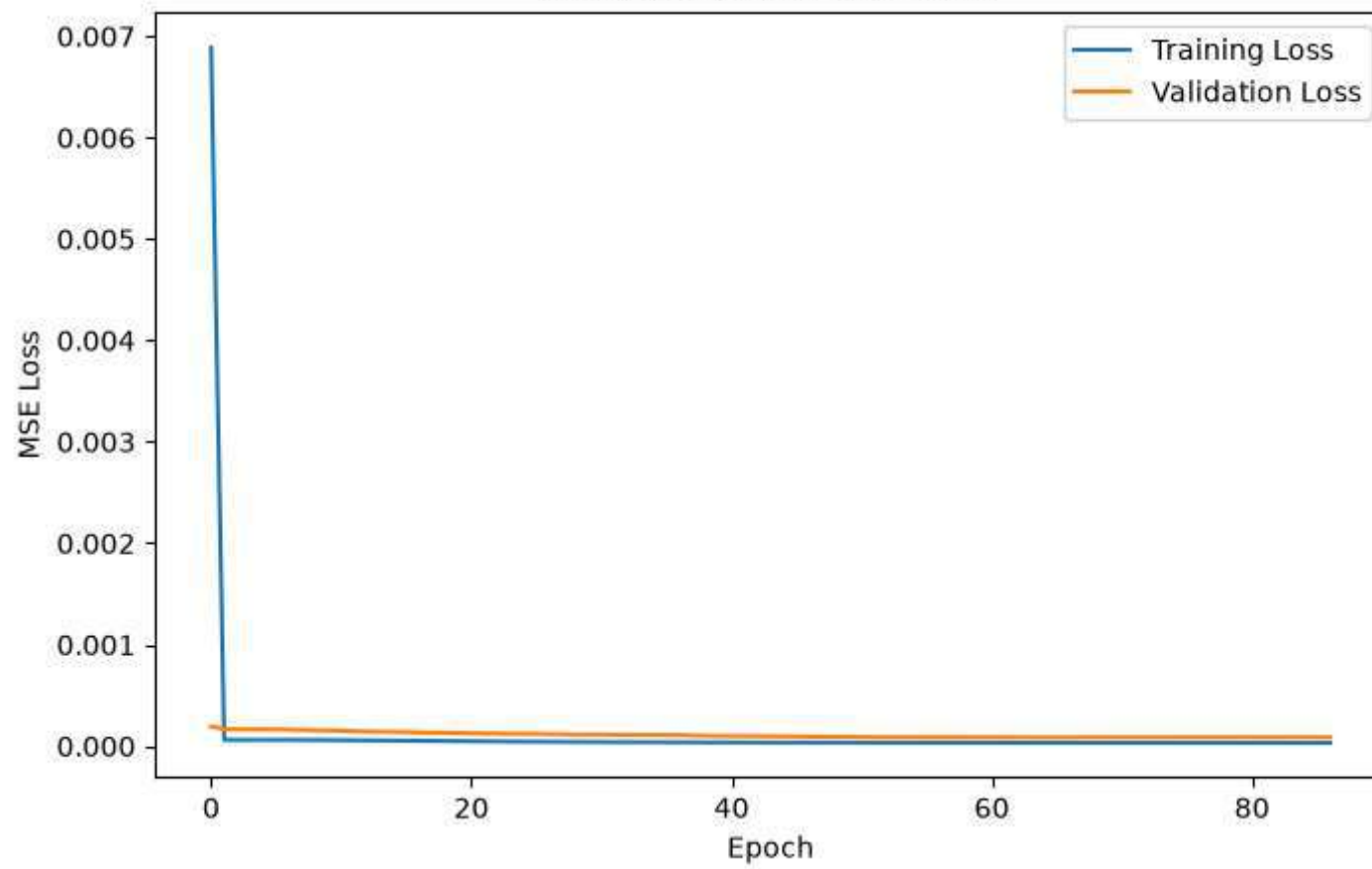
Epoch 64/120
406/406  1s 3ms/step - loss: 3.2129e-05 - val_loss: 8.4846e-05
Epoch 65/120
406/406  1s 3ms/step - loss: 3.2057e-05 - val_loss: 8.4795e-05
Epoch 66/120
406/406  1s 3ms/step - loss: 3.1952e-05 - val_loss: 8.4715e-05
Epoch 67/120
406/406  1s 3ms/step - loss: 3.1924e-05 - val_loss: 8.4697e-05
Epoch 68/120
406/406  1s 3ms/step - loss: 3.1875e-05 - val_loss: 8.5424e-05
Epoch 69/120
406/406  1s 3ms/step - loss: 3.1831e-05 - val_loss: 8.5440e-05
Epoch 70/120
406/406  1s 3ms/step - loss: 3.1763e-05 - val_loss: 8.5622e-05
Epoch 71/120
406/406  1s 3ms/step - loss: 3.1771e-05 - val_loss: 8.5366e-05
Epoch 72/120
406/406  1s 3ms/step - loss: 3.1729e-05 - val_loss: 8.5812e-05
Epoch 73/120
406/406  1s 3ms/step - loss: 3.1688e-05 - val_loss: 8.5567e-05
Epoch 74/120
406/406  1s 3ms/step - loss: 3.1677e-05 - val_loss: 8.6026e-05
Epoch 75/120
406/406  1s 3ms/step - loss: 3.1621e-05 - val_loss: 8.5341e-05
Epoch 76/120
406/406  1s 3ms/step - loss: 3.1607e-05 - val_loss: 8.6137e-05
Epoch 77/120
406/406  1s 4ms/step - loss: 3.1594e-05 - val_loss: 8.6365e-05
Epoch 78/120
406/406  1s 3ms/step - loss: 3.1553e-05 - val_loss: 8.6485e-05
Epoch 79/120
406/406  1s 3ms/step - loss: 3.1589e-05 - val_loss: 8.5843e-05
Epoch 80/120
406/406  1s 3ms/step - loss: 3.1556e-05 - val_loss: 8.6670e-05
Epoch 81/120
406/406  1s 3ms/step - loss: 3.1499e-05 - val_loss: 8.6057e-05
Epoch 82/120
406/406  1s 3ms/step - loss: 3.1485e-05 - val_loss: 8.6187e-05
Epoch 83/120
406/406  1s 3ms/step - loss: 3.1486e-05 - val_loss: 8.6356e-05
Epoch 84/120
406/406  1s 3ms/step - loss: 3.1464e-05 - val_loss: 8.6508e-05

Epoch 85/120
406/406 ————— 1s 3ms/step - loss: 3.1446e-05 - val_loss: 8.6479e-05
Epoch 86/120
406/406 ————— 1s 3ms/step - loss: 3.1443e-05 - val_loss: 8.6511e-05
Epoch 87/120
406/406 ————— 1s 4ms/step - loss: 3.1437e-05 - val_loss: 8.5457e-05

In [43]: *# Plot IBM Modified V2 Loss*

```
plt.figure(figsize=(8, 5))
plt.plot(history_modified_v2_ibm.history["loss"], label="Training Loss")
plt.plot(history_modified_v2_ibm.history["val_loss"], label="Validation Loss")
plt.title("IBM Modified LSTM V2 Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.show()
```

IBM Modified LSTM V2 Loss



Berdasarkan grafik loss Modified LSTM V2 pada saham IBM, training loss dan validation loss berada pada nilai yang rendah dan relatif stabil. Training loss berada di sekitar $2.9e-05$, sedangkan validation loss berada di sekitar $8.5e-05$. Hal ini menunjukkan bahwa model mampu mempelajari pola data IBM dengan cukup stabil.

Model tidak berjalan sampai epoch maksimum karena menggunakan EarlyStopping. Training berhenti lebih awal karena validation loss tidak lagi menunjukkan perbaikan yang signifikan dalam beberapa epoch. Kondisi ini menunjukkan bahwa EarlyStopping bekerja sesuai tujuan, yaitu menghentikan training ketika performa pada validation set sudah tidak membaik.

Meskipun validation loss lebih tinggi dibandingkan training loss, gap tersebut masih relatif stabil. Oleh karena itu, Modified LSTM V2 tetap layak digunakan sebagai model hasil tuning dan akan dibandingkan dengan baseline menggunakan RMSE, MAE, dan MAPE pada test set.

```
In [44]: # Overfitting Check Function

def check_overfitting(history, model_name, stock_name):
    train_loss = history.history["loss"]
    val_loss = history.history["val_loss"]

    final_train_loss = train_loss[-1]
    final_val_loss = val_loss[-1]
    best_val_loss = min(val_loss)
    best_epoch = val_loss.index(best_val_loss) + 1

    gap = final_val_loss - final_train_loss
    gap_ratio = final_val_loss / final_train_loss

    if final_val_loss > final_train_loss and gap_ratio > 2:
        status = "Indication of overfitting"
    elif final_train_loss > final_val_loss and final_train_loss > 0.01:
        status = "Indication of underfitting"
    else:
        status = "No serious overfitting"

    return {
        "Stock": stock_name,
        "Model": model_name,
        "Final Train Loss": final_train_loss,
        "Final Validation Loss": final_val_loss,
        "Best Validation Loss": best_val_loss,
```

```

        "Best Epoch": best_epoch,
        "Loss Gap": gap,
        "Gap Ratio": gap_ratio,
        "Overfitting Status": status
    }

overfit_results = pd.DataFrame([
    check_overfitting(history_baseline_fb, "Baseline LSTM", "FB"),
    check_overfitting(history_modified_fb, "Modified LSTM", "FB"),
    check_overfitting(history_modified_v2_fb, "Modified LSTM V2", "FB"),
    check_overfitting(history_baseline_ibm, "Baseline LSTM", "IBM"),
    check_overfitting(history_modified_ibm, "Modified LSTM", "IBM"),
    check_overfitting(history_modified_v2_ibm, "Modified LSTM V2", "IBM")
])

overfit_results

```

Out[44]:

	Stock	Model	Final Train Loss	Final Validation Loss	Best Validation Loss	Best Epoch	Loss Gap	Gap Ratio	Overfitting Status
0	FB	Baseline LSTM	0.000178	0.001165	0.001165	50	0.000987	6.542245	Indication of overfitting
1	FB	Modified LSTM	0.001991	0.001213	0.001144	4	-0.000778	0.609155	No serious overfitting
2	FB	Modified LSTM V2	0.000109	0.000621	0.000621	150	0.000512	5.684526	Indication of overfitting
3	IBM	Baseline LSTM	0.000032	0.000087	0.000085	39	0.000055	2.706518	Indication of overfitting
4	IBM	Modified LSTM	0.000086	0.000209	0.000128	42	0.000124	2.444002	Indication of overfitting
5	IBM	Modified LSTM V2	0.000031	0.000085	0.000085	67	0.000054	2.718340	Indication of overfitting

Training Behavior and Overfitting Check

Tabel overfitting check digunakan sebagai analisis tambahan untuk melihat perbandingan antara training loss dan validation loss pada setiap model. Analisis ini membantu melihat apakah terdapat gap antara performa model pada train set dan validation set.

Berdasarkan hasil tersebut, beberapa model menunjukkan validation loss yang lebih tinggi dibandingkan training loss. Hal ini wajar pada data time series karena model biasanya lebih mudah mempelajari train set dibandingkan data validation yang berada pada periode waktu berbeda.

Pada saham FB, Modified LSTM V2 tetap menunjukkan gap antara training loss dan validation loss, tetapi validation loss yang diperoleh lebih rendah dibandingkan baseline dan modified model pertama. Pada saham IBM, Modified LSTM V2 juga memiliki validation loss yang sedikit lebih baik dibandingkan baseline.

Oleh karena itu, tabel ini digunakan sebagai analisis tambahan terhadap perilaku training, bukan sebagai satu-satunya dasar pemilihan model. Penentuan model terbaik tetap dilakukan berdasarkan evaluasi pada test set menggunakan RMSE, MAE, dan MAPE.

1d. Model Evaluation

Pada bagian ini dilakukan evaluasi terhadap baseline LSTM, modified LSTM, dan Modified LSTM V2 pada test set. Test set yang digunakan merupakan periode satu tahun terakhir, sehingga data ini tidak digunakan pada proses training maupun validation.

Metrik evaluasi yang digunakan adalah RMSE, MAE, dan MAPE. RMSE digunakan untuk melihat besar error dengan penalti lebih besar pada kesalahan prediksi yang tinggi. MAE menunjukkan rata-rata selisih absolut antara nilai aktual dan prediksi. MAPE menunjukkan rata-rata persentase error terhadap nilai aktual.

Karena model dilatih menggunakan data yang sudah dinormalisasi, hasil prediksi perlu dikembalikan terlebih dahulu ke skala harga asli menggunakan inverse transform dari MinMaxScaler.

```
In [45]: # Evaluation Function

def evaluate_lstm_model(model, X_test, y_test, scaler):
    y_pred_scaled = model.predict(X_test)

    y_actual = scaler.inverse_transform(y_test.reshape(-1, 1))
    y_pred = scaler.inverse_transform(y_pred_scaled)
```

```

rmse = np.sqrt(mean_squared_error(y_actual, y_pred))
mae = mean_absolute_error(y_actual, y_pred)
mape = np.mean(np.abs((y_actual - y_pred) / y_actual)) * 100

return rmse, mae, mape, y_actual, y_pred

```

In [46]: *# FB Evaluation*

```

fb_baseline_rmse, fb_baseline_mae, fb_baseline_mape, fb_actual, fb_baseline_pred = evaluate_lstm_model(
    baseline_fb, X_test_fb, y_test_fb, fb_scaler
)

fb_modified_rmse, fb_modified_mae, fb_modified_mape, _, fb_modified_pred = evaluate_lstm_model(
    modified_fb, X_test_fb, y_test_fb, fb_scaler
)

fb_modified_v2_rmse, fb_modified_v2_mae, fb_modified_v2_mape, _, fb_modified_v2_pred = evaluate_lstm_model(
    modified_v2_fb, X_test_fb, y_test_fb, fb_scaler
)

# IBM Evaluation

ibm_baseline_rmse, ibm_baseline_mae, ibm_baseline_mape, ibm_actual, ibm_baseline_pred = evaluate_lstm_model(
    baseline_ibm, X_test_ibm, y_test_ibm, ibm_scaler
)

ibm_modified_rmse, ibm_modified_mae, ibm_modified_mape, _, ibm_modified_pred = evaluate_lstm_model(
    modified_ibm, X_test_ibm, y_test_ibm, ibm_scaler
)

ibm_modified_v2_rmse, ibm_modified_v2_mae, ibm_modified_v2_mape, _, ibm_modified_v2_pred = evaluate_lstm_model(
    modified_v2_ibm, X_test_ibm, y_test_ibm, ibm_scaler
)

```

```

8/8 ————— 0s 5ms/step
8/8 ————— 0s 3ms/step
8/8 ————— 0s 27ms/step
8/8 ————— 0s 4ms/step
8/8 ————— 0s 5ms/step
8/8 ————— 0s 25ms/step

```

In [47]: *# Evaluation Result Table*

```
evaluation_result = pd.DataFrame({
    "Stock": ["FB", "FB", "FB", "IBM", "IBM", "IBM"],
    "Model": [
        "Baseline LSTM", "Modified LSTM", "Modified LSTM V2",
        "Baseline LSTM", "Modified LSTM", "Modified LSTM V2"
    ],
    "RMSE": [
        fb_baseline_rmse, fb_modified_rmse, fb_modified_v2_rmse,
        ibm_baseline_rmse, ibm_modified_rmse, ibm_modified_v2_rmse
    ],
    "MAE": [
        fb_baseline_mae, fb_modified_mae, fb_modified_v2_mae,
        ibm_baseline_mae, ibm_modified_mae, ibm_modified_v2_mae
    ],
    "MAPE (%)": [
        fb_baseline_mape, fb_modified_mape, fb_modified_v2_mape,
        ibm_baseline_mape, ibm_modified_mape, ibm_modified_v2_mape
    ]
})

evaluation_result
```

Out[47]:

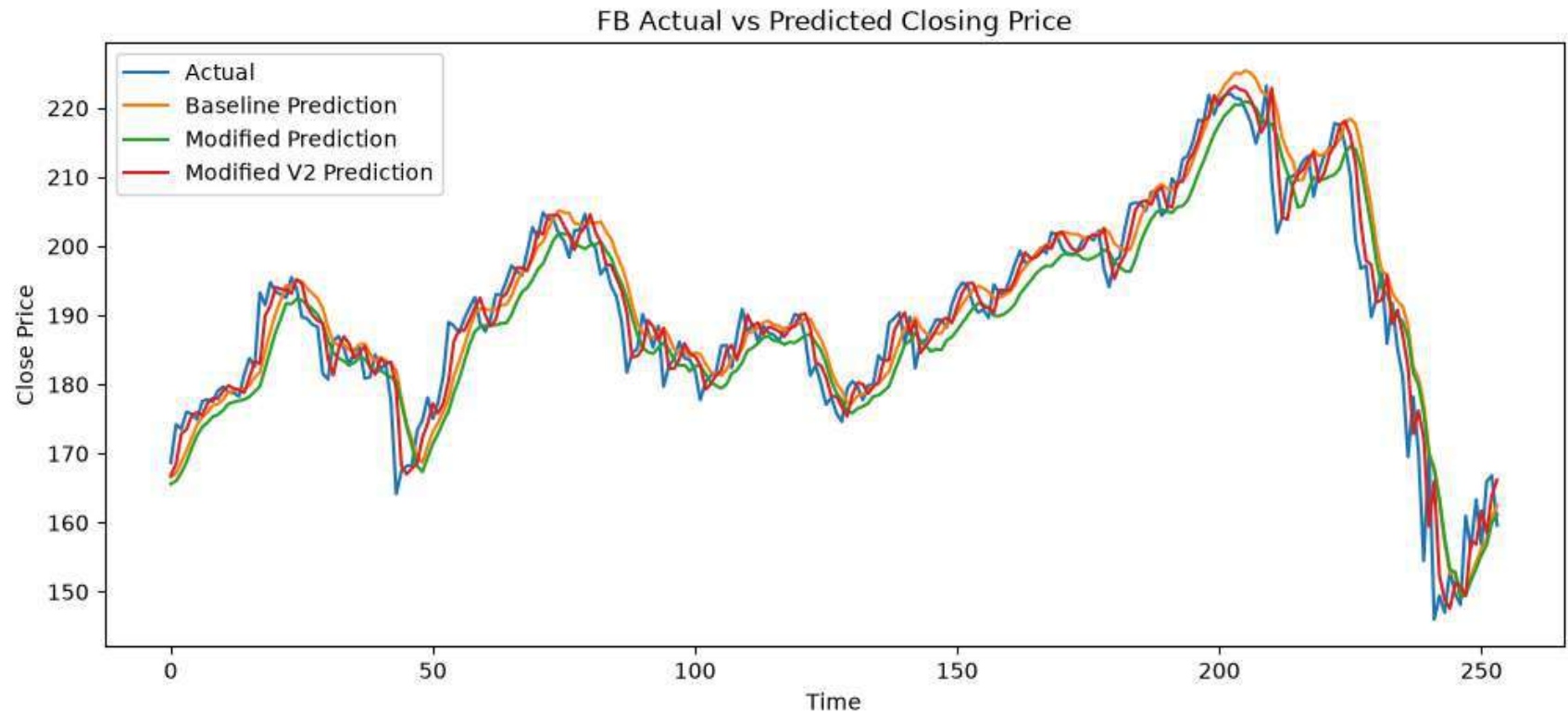
	Stock	Model	RMSE	MAE	MAPE (%)
0	FB	Baseline LSTM	5.549302	3.890852	2.103762
1	FB	Modified LSTM	5.595039	4.320164	2.316825
2	FB	Modified LSTM V2	4.057168	2.788925	1.511659
3	IBM	Baseline LSTM	2.674839	1.785442	1.373415
4	IBM	Modified LSTM	3.111014	2.058484	1.572655
5	IBM	Modified LSTM V2	2.652227	1.709225	1.317668

In [48]: *# FB Actual vs Predicted*

```
plt.figure(figsize=(12, 5))
```

```
plt.plot(fb_actual, label="Actual")
plt.plot(fb_baseline_pred, label="Baseline Prediction")
plt.plot(fb_modified_pred, label="Modified Prediction")
plt.plot(fb_modified_v2_pred, label="Modified V2 Prediction")

plt.title("FB Actual vs Predicted Closing Price")
plt.xlabel("Time")
plt.ylabel("Close Price")
plt.legend()
plt.show()
```



Actual vs Predicted Result for FB

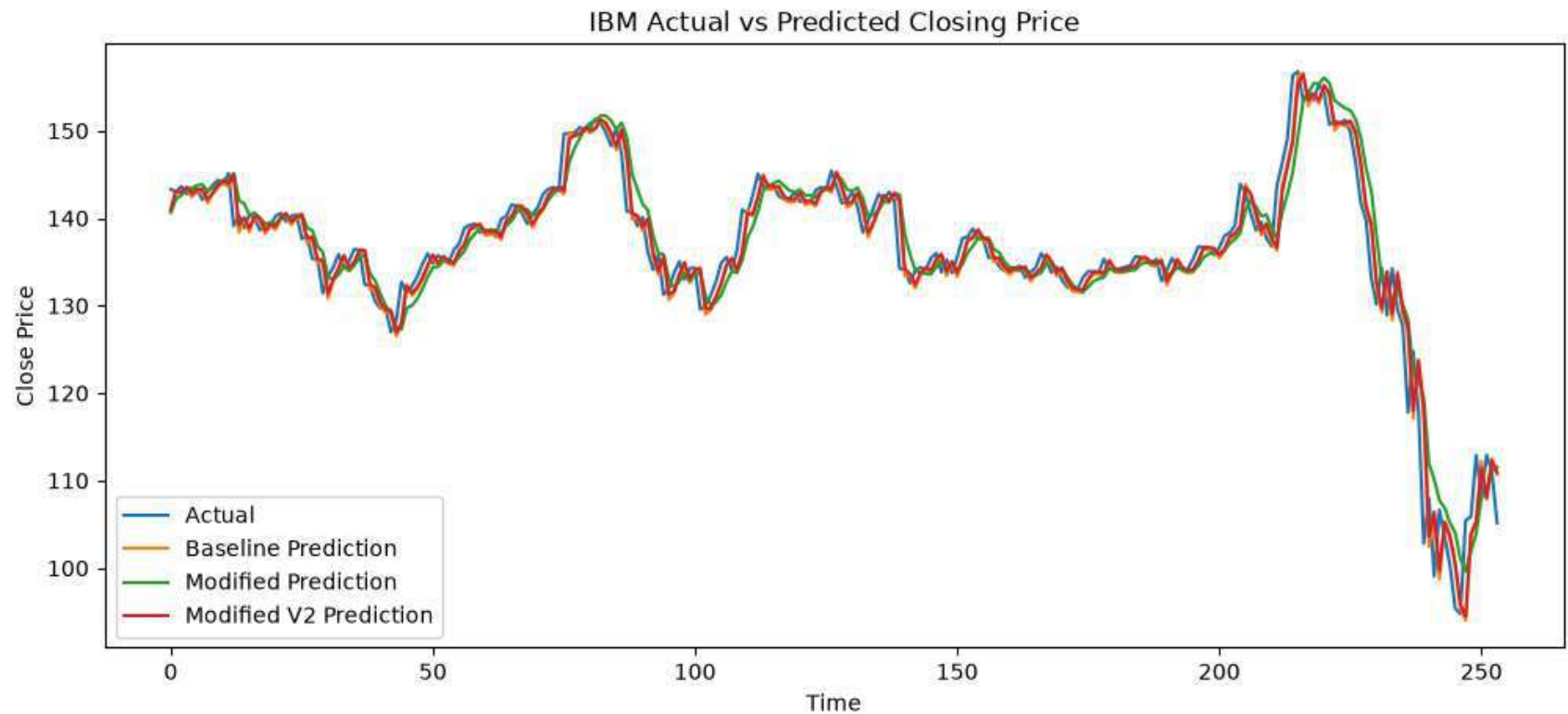
Pada saham FB, grafik actual vs predicted menunjukkan bahwa model LSTM mampu mengikuti pola utama harga penutupan aktual pada test set satu tahun terakhir. Modified LSTM V2 terlihat paling dekat dengan pola aktual dibandingkan baseline LSTM dan modified model pertama.

Berdasarkan hasil evaluasi numerik, Modified LSTM V2 memberikan performa terbaik pada saham FB. Model ini memperoleh RMSE sebesar 4.057168, MAE sebesar 2.788925, dan MAPE sebesar 1.511659%. Nilai tersebut lebih rendah dibandingkan baseline LSTM dan modified LSTM pertama, sehingga Modified LSTM V2 lebih akurat dalam memprediksi harga penutupan saham FB pada test set.

In [49]: *# IBM Actual vs Predicted*

```
plt.figure(figsize=(12, 5))
plt.plot(ibm_actual, label="Actual")
plt.plot(ibm_baseline_pred, label="Baseline Prediction")
plt.plot(ibm_modified_pred, label="Modified Prediction")
plt.plot(ibm_modified_v2_pred, label="Modified V2 Prediction")

plt.title("IBM Actual vs Predicted Closing Price")
plt.xlabel("Time")
plt.ylabel("Close Price")
plt.legend()
plt.show()
```



Actual vs Predicted Result for IBM

Pada saham IBM, grafik actual vs predicted menunjukkan bahwa baseline LSTM dan Modified LSTM V2 sama-sama mampu mengikuti pola pergerakan harga aktual dengan cukup baik. Namun, Modified LSTM V2 menghasilkan nilai error yang paling rendah pada test set.

Berdasarkan hasil evaluasi numerik, Modified LSTM V2 memberikan performa terbaik pada saham IBM. Model ini memperoleh RMSE sebesar 2.652227, MAE sebesar 1.709225, dan MAPE sebesar 1.317668%. Nilai tersebut lebih rendah dibandingkan baseline LSTM dan modified LSTM pertama, sehingga Modified LSTM V2 dipilih sebagai model terbaik untuk saham IBM.

Evaluation Summary

Berdasarkan hasil evaluasi pada test set satu tahun terakhir, Modified LSTM V2 memberikan performa terbaik pada saham FB dan IBM.

Pada saham FB, baseline LSTM memperoleh RMSE sebesar 5.549302, MAE sebesar 3.890852, dan MAPE sebesar 2.103762%. Modified LSTM pertama memperoleh RMSE sebesar 5.595039, MAE sebesar 4.320164, dan MAPE sebesar 2.316825%. Setelah dilakukan tuning lanjutan, Modified LSTM V2 memperoleh hasil terbaik dengan RMSE sebesar 4.057168, MAE sebesar 2.788925, dan MAPE sebesar 1.511659%.

Pada saham IBM, baseline LSTM memperoleh RMSE sebesar 2.674839, MAE sebesar 1.785442, dan MAPE sebesar 1.373415%. Modified LSTM pertama memperoleh RMSE sebesar 3.111014, MAE sebesar 2.058484, dan MAPE sebesar 1.572655%. Modified LSTM V2 memberikan hasil terbaik dengan RMSE sebesar 2.652227, MAE sebesar 1.709225, dan MAPE sebesar 1.317668%.

Karena RMSE, MAE, dan MAPE merupakan metrik error, nilai yang lebih kecil menunjukkan performa yang lebih baik. Dengan demikian, Modified LSTM V2 dipilih sebagai model terbaik untuk prediksi harga penutupan saham FB dan IBM.